

xrtc-server 跨模块横向深潜

拆掉模块之间的墙。按数据流链路重新编排。

配套：系统总览 [README.md](#)、模块纵向深潜 [DEEP-DIVE.md](#)。

目录

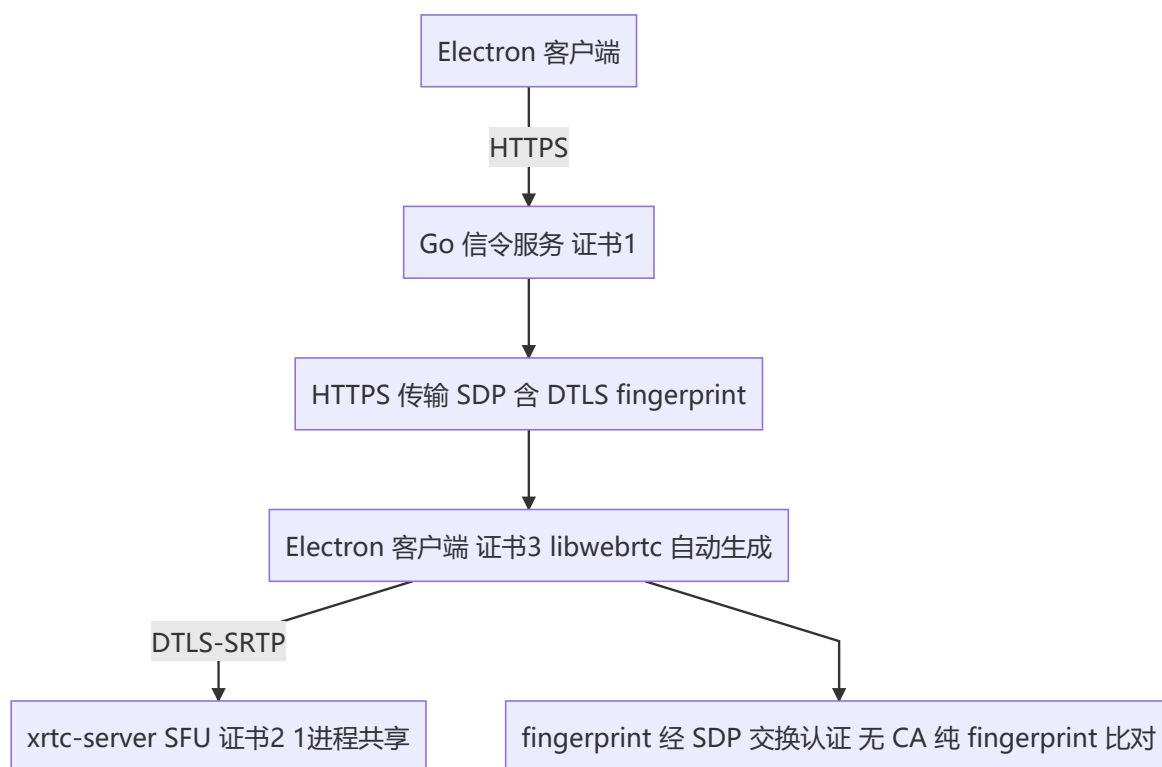
1. [1. 证书体系：HTTPS + DTLS 双层认证](#)
 - [1.1 全景](#)
 - [1.2 证书 ①：HTTPS 证书 \(Go 信令服务\)](#)
 - [1.3 证书 ②：DTLS 证书 \(xrtc-server SFU\)](#)
 - [1.4 证书 ③：DTLS 证书 \(客户端\)](#)
 - [1.5 DTLS 握手过程中的证书交换](#)
 - [1.6 为什么不需要 CA? fingerprint 机制](#)
 - [1.7 总结](#)
2. [2. TCP 信令 vs UDP 媒体：同一 libev LT 下的两种 I/O 哲学](#)
 - [2.1 读策略](#)
 - [2.2 写策略](#)
 - [2.3 汇总对比](#)
3. [3. set local description：整个媒体栈的"接生婆"](#)
 - [3.1 入口时间线](#)
 - [3.2 create_offer\(\) 的四个阶段](#)
 - [3.3 TransportController::set_local_description\(\) 的 8 个副作用](#)
 - [3.4 gathering_candidate\(\) → UDP socket 绑定](#)
4. [4. Candidate Pair → IceConnection：从 UDP 四元组到逻辑通道](#)
 - [4.5 IceTransportChannel × Network → UDPPort 的数量关系](#)
 - [4.6 Candidate Pair → IceConnection：从 UDP 四元组到逻辑通道](#)
5. [5. ICE ping 定时器 + 两层限速 + 5 级排序](#)
 - [5.7 ICE ping 定时器的冷启动条件](#)
 - [5.8 定时器的三职责 + 两层限速](#)
 - [5.1 关键常量速查](#)
 - [5.2 冷启动：从第一个 UDP 包到定时器启动](#)
 - [5.3 稳态：on_check_and_ping 每周循环](#)
 - [5.4 第一层限速：Channel 级速率门](#)
 - [5.5 第二层限速：Connection 级间隔 + Round-Robin](#)
 - [5.6 两层限速完整矩阵](#)
 - [5.7 连接写状态降级：update_state](#)
 - [5.8 5 级排序算法 + RTT 防抖](#)
 - [5.9 RTT 指数平滑](#)
 - [5.10 write_state 超时 → k failed → UAF 防护链](#)
 - [5.11 状态变化驱动的重排序路径](#)
 - [5.12 完整调用链总览](#)
6. [6. ICE 四层状态机](#)
 - [6.9 ICE 状态机：从 IceConnection 到 IceTransportChannel 到 IceAgent](#)
7. [7. SDP 字段 → ICE/DTLS 的"最后一公里"](#)
 - [7.1 字段提取](#)

- 7.2 [分发：同一个函数同时喂 ICE 和 DTLS](#)
- 7.3 [ice-ufrag/pwd 的两把密码 + 各自消费点](#)
- 7.4 [密码的"先有鸡还是先有蛋"问题](#)
- 7.5 [fingerprint 的两个时序路径](#)
- 8. [8. 三包竞赛：STUN / DTLS ClientHello / ANSWER 时序全集](#)
 - 8.1 [六种时序](#)
 - 8.2 [时序 4 详细走读（唯一有延迟损失的路径）](#)
 - 8.3 [caught_client_hello 的真实作用范围](#)
- 9. [9. 同一个 UDP socket 如何服务三层协议](#)
 - 9.1 [收包分拣链（完整）](#)
 - 9.2 [四个分拣层](#)
 - 9.3 [发包容积链（完整）](#)
- 10. [10. 信号链：ICE writable → DTLS → SRTP → RTP 就绪](#)
 - 10.1 [信号订阅全景](#)
 - 10.2 [多米诺骨牌](#)
 - 10.3 [PC 状态聚合](#)
- 11. [11. RTP/RTCP 数据包处理](#)
 - 11.1 [两级解复用](#)
 - 11.2 [解密与转发链](#)
 - 11.3 [RtcStreamManager 转发逻辑](#)
 - 11.4 [发包路径](#)
- 12. [12. PULL 流 + SSRC 透传](#)
 - 12.1 [创建流程](#)
 - 12.2 [PushStream 如何提取 SSRC](#)
 - 12.3 [PullStream 的 offer 怎么用这些 SSRC](#)
 - 12.4 [PushStream vs PullStream 对比](#)
- 13. [13. STOP 与资源清理链](#)
 - 13.1 [两个清理入口](#)
 - 13.2 [UID 校验 + delete](#)
 - 13.3 [析构瀑布](#)
 - 13.4 [为什么需要 10ms 延迟析构](#)
 - 13.5 [STOP 命令 vs 异常清理对比](#)
- 14. [14. 完整生命周期时间线](#)
- 15. [15. FAQ：你应该知道但可能没问的问题](#)
 - 15.1 [STUN MESSAGE-INTEGRITY 的密码使用规则："发给谁，就用谁的密码"](#)
 - 15.2 [DtlsTransport::send_packet\(\) 为什么绕过 DTLS 加密？](#)
 - 15.3 [30s ICE 超时和 PC 状态 k_connected 的关系](#)
 - 15.4 [PushStream 和 PullStream 的 create_offer\(\) 本质区别](#)
 - 15.5 [DtlsTransport 和 DtlsSrtpTransport 的"组合优于继承"关系](#)
 - 15.6 [PeerConnection::destroy\(\) 延迟析构](#)
 - 15.7 [StreamInterfaceChannel 的 BIO 桥接](#)

1. 证书体系：HTTPS + DTLS 双层认证

一个完整的 xrtc-server + Electron 客户端推拉流系统，涉及 **3 个证书**，分属两层。

1.1 全景



1.2 证书 ①：HTTPS 证书（Go 信令服务）

- **谁持有：**Go 信令服务
- **数量：**1 个/进程
- **生成方式：**开发自签名（`openssl req -x509`），生产 CA 签发
- **文件：**参考工程 `conf/fullchain.pem` + `conf/privkey.pem`
- **用途：**客户端通过 HTTPS 连接信令服务，SDP offer/answer 经加密信道传输

Electron 客户端 — HTTPS (TLS 1.2/1.3) —> Go 信令服务 (:8081)
证书 ① 公钥加密 证书 ① 私钥解密

为什么 HTTPS 必不可少？ SDP 中包含 DTLS fingerprint——这个值决定了后续媒体层的身份认证。如果信令通道被中间人攻击，攻击者可以替换 SDP 中的 fingerprint，从而在 DTLS 握手时冒充对端。HTTPS 保证了 fingerprint 从发出到接收全程不被篡改。

1.3 证书 ②：DTLS 证书（xrtc-server SFU）

- **谁持有：**xrtc-server
- **数量：**1 个/xrtc-server 进程，所有 PushStream/PullStream 共享
- **生成方式：**`rtc::RTCCertificateGenerator::GenerateCertificate()` 自签名，有效期 1 年

生成代码（`src/server/rtc_server.cpp:55-73`）：

```
int RtcServer::_generate_and_check_certificate() {
    if (!_certificate || !_certificate->HasExpired(time(NULL) * 1000)) {
        rtc::KeyParams key_params;
        _certificate = rtc::RTCCertificateGenerator::GenerateCertificate(
            key_params, k_year_in_ms); // 自签名, 365 天有效
    }
    return 0;
}
```

共享方式——每条消息携带同一个证书指针 (src/server/rtc_server.cpp:225) :

```
// _process_rtc_msg() 中
msg->certificate = _certificate.get(); // 所有流共用一个证书
```

传递链: RtcServer → RtcMsg::certificate → RtcWorker::_process_push/pull →
RtcStreamManager::create_push/pull_stream → RtcStream::start →
PeerConnection::init → TransportController::set_local_certificate →
DtlsTransport::set_local_certificate

为什么可以共享? DTLS 认证不走 CA 链, 走 SDP fingerprint 比对。证书只是公私钥对的容器, 复用同一对密钥不影响安全性——每个流独立的 DTLS 会话由各自的 SSLStreamAdapter 实例管理, handshake 时各自生成独立的 session key, 不会跨流泄露。

fingerprint 如何进入 SDP: PeerConnection::create_offer() 中调用
SessionDescription::add_transport_info(mid, ice_params, _certificate), 内部计算
_certificate->GetSSLCertificate().GetFingerprint("sha-256") 并写入
TransportDescription::identity_fingerprint, 最终序列化为 SDP 的 a=fingerprint:sha-256
AA:BB:CC:... 行。

1.4 证书 ③: DTLS 证书 (客户端)

- **谁持有:** Electron 客户端 (libwebrtc 内部)
- **数量:** 1 个/RTCPeerConnection 实例
- **生成方式:** libwebrtc 在 PeerConnection 构造时自动生成 RSA/ECDSA 密钥对 + 自签名证书
- **开发者不需要写任何代码:** 完全由 libwebrtc 内部管理

客户端的 fingerprint 写入 SDP answer 的 a=fingerprint:sha-256 xx:yy:zz:... 行。SFU 收到 answer 后解析 → DtlsTransport::set_remote_fingerprint() → SetPeerCertificateDigest()。

1.5 DTLS 握手过程中的证书交换

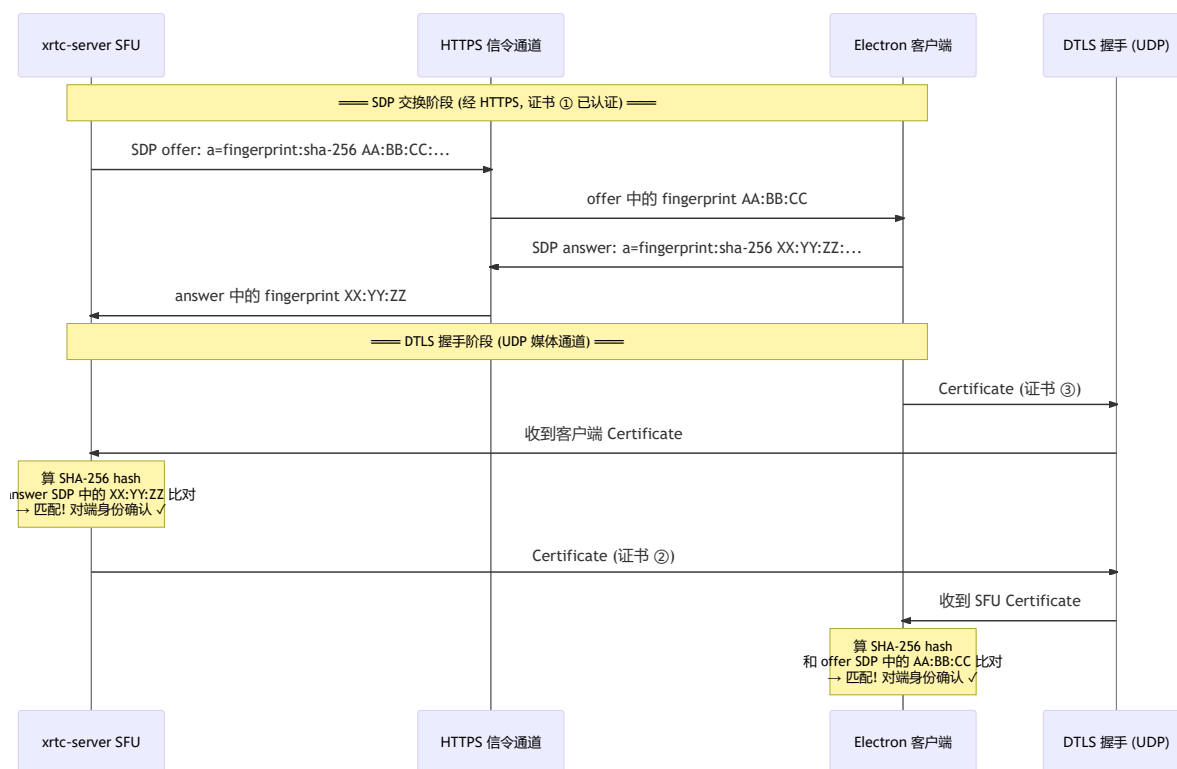
DTLS 1.2 完整的握手消息序列 (与 TLS 1.2 相同, 加了 cookie) :



为什么 ClientHello 里没有证书？ 因为 DTLS 握手用的是标准 TLS 1.2 握手流程——Certificate 消息是独立的握手消息（类型 11），在 ServerHello 之后、ClientKeyExchange 之前。ClientHello 只包含随机数、密码套件列表、Session ID、支持的扩展等，不包含证书。

1.6 为什么不需要 CA? fingerprint 机制

这是 WebRTC DTLS 设计最精妙的地方：



信任链：

1. SDP 走 HTTPS → fingerprint 值被证书 ① 保护，不可能被中间人篡改

- 2. DTLS 握手时收到对端证书 → hash 比对 → 匹配即证明"你就是刚才通过 HTTPS 跟我协商 SDP 的那个人"
- 3. 自签名证书的一对一绑定完成，无需 CA 参与

RFC 8122 将这种模式称为 "Certificate Fingerprint"——SDP 中携带的 fingerprint 充当了"带外预共享的信任锚"，替代了 PKI 中 CA 的角色。

1.7 总结

证书	谁持有	数量	生成	认证方式	信任锚	用途
HTTPS	Go 信令服务	1/进程	自签名或 CA	TLS 标准 CA 链 / 自签名	CA 根证书	保护 SDP 传输
DTLS (SFU)	xrtc-server	1/进程	<code>RTCCertificateGenerator</code>	SDP offer fingerprint	HTTPS 信道	DTLS 握手 + SRTP 密钥
DTLS (客户端)	libwebrtc	1/PeerConnection	libwebrtc 自动生成	SDP answer fingerprint	HTTPS 信道	DTLS 握手 + SRTP 密钥

两层认证，三份证书：HTTPS（证书 ①）保护信令面的 SDP 交换 → SDP 中的 fingerprint 成为媒体面的信任锚 → DTLS 握手（证书 ②+③）通过 fingerprint 比对完成双方身份认证 → 握手主密钥派生 SRTP 加密密钥。

2. TCP 信令 vs UDP 媒体：同一 libev LT 下的两种 I/O 哲学

libev 只有 LT（水平触发）模式，但 TCP 和 UDP 在 LT 下的读写策略截然不同。核心原因是**字节流 vs 数据报**的本质差异。

2.1 读策略

TCP（`src/server/signaling_worker.cpp:328-360`）：每次 READ 事件只做一次 `read()`，读取精确的 `bytes_expected` 字节（36 或 `body_len`）。

```
void SignalingWorker::_read_query(int fd) {
    int read_len = c->bytes_expected;           // 精确期望字节数
    int qb_len = sdslen(c->querybuf);           // 已有数据偏移
    c->querybuf = sdsMakeRoomFor(c->querybuf, read_len);
    nread = sock_read_data(fd, c->querybuf + qb_len, read_len); // 一次 read()
    sdsIncrLen(c->querybuf, nread);
    _process_query_buffer(c);                    // 尝试推进状态机
}
```

为什么不循环读到 EAGAIN？ 因为 `bytes_expected` 精准限定了当前阶段需要多少字节。多余数据属于下一个请求（短连接下不存在）。LT 保证：如果内核缓冲还有数据而这次没读完，libev 会**立即重新触发**。

UDP（`src/base/async_udp_socket.cpp:58-76`）：每次 READ 事件**循环** `recvfrom()` 直到 EAGAIN。

```

void AsyncUdpSocket::recv_data() {
    while (true) {
        int len = sock_recvfrom(_socket, _buf, _size, ...);
        if (len <= 0) return; // EAGAIN → 退出循环
        signal_read_packet(this, _buf, len, remote_addr, ts);
    }
}

```

为什么要循环？ UDP 是数据报，一个 `recvfrom` 只取一个。如果网络突发导致内核缓冲积压了多个数据报，LT 会对每个剩余数据报各触发一次 `epoll_wait` → 回调。while 循环一次性排空，避免多余系统调用。**本质是用 ET 风格跑在 LT 之上。**

2.2 写策略

TCP (src/server/signaling_worker.cpp:362-393) :

- 写事件**按需启用**： `_add_reply` 才 `start_io_event(WRITE)`，发完立即 `stop_io_event(WRITE)`
- **支持部分写**： `write()` 可能只发部分字节，用 `cur_resp_pos` 跟踪偏移，LT 重触发时继续
- **致命错误**： `write()` 返回 -1 → `_close_conn`

UDP (src/base/async_udp_socket.cpp:129-163) :

- 先乐观 `sendto()`，失败再入队 + 启用 WRITE
- **不支持部分写**： `sendto` 要么全发要么全不发（数据报原子性）
- **写缓冲区满不是致命错误**：入队等待下次触发，不关闭 socket

2.3 汇总对比

维度	TCP 信令	UDP 媒体
读策略	一次 <code>read()</code> 读精确字节数，靠 LT 重触发	<code>while(true)</code> 循环到 EAGAIN
读粒度	字节流，需 sds 累积 + 状态机拼包	数据报，每包完整独立
状态机	STATE_HEAD → STATE_BODY	无状态机（上层分拣）
写启用	按需启用，发完关闭	按需启用，队列空关闭
部分写	支持（ <code>cur_resp_pos</code> 跟踪偏移）	不支持（数据报原子性）
写失败	<code>write()</code> -1 → 关闭连接	<code>sendto()</code> 0 → 入队等待
连接生命周期	短连接（ <code>bytes_processed=65536</code> ）	长连接（ICE 会话期间）
反压	TCP 流控 + <code>reply_list</code>	<code>_udp_packet_list</code>

3. set_local_description：整个媒体栈的"接生婆"

`set_local_description` 不只是往 SDP 里填 candidate。它创建了 ICE channel、UDP socket、DTLS/SRTP 传输对象、以及连接这一切的信号链。它是整个媒体栈的出生证明。

3.1 入口时间线

```
RtcWorker::_process_push(msg)
→ RtcStreamManager::create_push_stream(uid, stream_name, ...)
→ new PushStream(...)
→ stream->start(certificate)           // ① 初始化 PC + 启动 30s 超时定时器
→ offer = stream->create_offer()       // ② 媒体栈诞生 + 返回 SDP 文本
→ _push_streams[stream_name] = stream
```

start() 做两件事 (src/stream/rtc_stream.cpp:81-85) :

```
void RtcStream::start(rtc::RTCCertificate* certificate) {
    _ice_timeout_watcher = _el->create_timer(ice_timeout_cb, this, false);
    _el->start_timer(_ice_timeout_watcher, k_ice_timeout * 1000); // 30s 兜底超时
    _pc->init(certificate); // 传递 DTLS 证书到 TransportController
}
```

30s 超时含义: 如果 30s 内 PC 状态没到 k_connected → 触发 on_stream_exception → 清理流资源。是整个系统的兜底超时。

3.2 create_offer() 的四个阶段

PeerConnection::create_offer() (src/pc/peer_connection.cpp:99-151) :

阶段 A — 随机生成 ICE 凭据 (line 107) :

```
IceParameters ice_params = IceCredentials::create_random_ice_credentials();
```

生成的 ice-ufrag 和 ice-pwd 被后续三个地方消费: SDP 文本、STUN MESSAGE-INTEGRITY HMAC 计算、UDPPort 的身份标识。

阶段 B — 构造 SDP 对象模型 (lines 109-146) :

- 创建 AudioContentDescription + VideoContentDescription
- add_transport_info(mid, ice_params, certificate) 把 ufrag/pwd/fingerprint 写入 TransportDescription
- BUNDLE group: audio + video 共享同一传输通道

阶段 C — set_local_description() (line 148) : 整个媒体栈的出生点, 见 §1.3。

阶段 D — 序列化为 SDP 文本 (line 150) : candidate 信息已通过信号回调填入, SDP 含 a=candidate:... 行。

3.3 TransportController::set_local_description() 的 8 个副作用

(src/pc/transport_controller.cpp:35-82)

```
int TransportController::set_local_description(SessionDescription* desc) {
    for (auto content : desc->contents()) {
        std::string mid = content->mid();

        // BUNDLE: audio + video 共享同一 ICE/DTLS 通道
    }
}
```

```

        if (desc->is_bundle(mid) && mid != desc->get_first_bundle_mid())
            continue;

        // === 副作用 ①: 创建 ICE channel ===
        _ice_agent->create_channel(_el, mid, IceCandidateComponent::RTP);

        // === 副作用 ②: 设置本地 ICE 凭据 (ufrag + pwd) ===
        auto td = desc->get_transport_info(mid);
        _ice_agent->set_ice_params(mid, IceCandidateComponent::RTP,
            IceParameters(td->ice_ufrag, td->ice_pwd));

        // === 副作用 ③: 创建 DtlsTransport, 绑定到 ICE channel ===
        DtlsTransport* dtls_transport = new DtlsTransport(
            _ice_agent->get_channel(mid, IceCandidateComponent::RTP));
        dtls_transport->set_local_certificate(_local_certificate);

        // === 副作用 ④: 连接 DTLS 状态信号 → TransportController ===
        dtls_transport->signal_receiving_state.connect(this, ...);
        dtls_transport->signal_writable_state.connect(this, ...);
        dtls_transport->signal_dtls_state.connect(this, ...);

        // === 副作用 ⑤: 连接 ICE 状态信号 → TransportController ===
        _ice_agent->signal_ice_state.connect(this, ...);

        // === 副作用 ⑥: 创建 DtlsSrtpTransport ===
        DtlsSrtpTransport* dtls_srtp = new DtlsSrtpTransport(...);
        dtls_srtp->set_dtls_transport(dtls_transport, nullptr);

        // === 副作用 ⑦: 连接 SRTP 收包信号 → 上层 ===
        dtls_srtp->signal_rtp_packet_received.connect(this, ...);
        dtls_srtp->signal_rtcp_packet_received.connect(this, ...);
    }

    // === 副作用 ⑧: 触发 candidate 收集 (创建 UDP sockets!) ===
    _ice_agent->gathering_candidate();
    return 0;
}

```

3.4 gathering_candidate() → UDP socket 绑定

```

IceAgent::gathering_candidate()
→ for each channel: channel->gathering_candidate()
→ IceTransportChannel::gathering_candidate() (ice_transport_channel.cpp:85-121)

```

```

void IceTransportChannel::gathering_candidate() {
    if (_ice_params.ice_ufrag.empty() || _ice_params.ice_pwd.empty()) return;
    auto network_list = _allocator->get_networks();

    for (auto network : network_list) {
        UDPPort* port = new UDPPort(_el, _transport_name, _component,
            _ice_params);
        port->signal_unknown_address.connect(this,
            &IceTransportChannel::_on_unknown_address);

        Candidate c;
    }
}

```

```

        port->create_ice_candidate(network, _allocator->min_port(),
            _allocator->max_port(), c);
        _local_candidates.push_back(c);
    }
    signal_candidate_allocate_done(this, _local_candidates);
}

```

`UDPPort::create_ice_candidate()` (`src/ice/udp_port.cpp:41-97`) 的实际 socket 操作:

```

int UDPPort::create_ice_candidate(Network* network, int min_port, int max_port,
Candidate& c) {
    _socket = create_udp_socket(network->ip().family()); // socket()
    sock_setnonblock(_socket); // O_NONBLOCK
    sock_bind(_socket, ..., min_port, max_port); // bind()
    sock_get_address(_socket, nullptr, &port); // getsockname()

    _async_socket = std::make_unique<AsyncUdpSocket>(_el, _socket);
    _async_socket->signal_read_packet.connect(this, &UDPPort::_on_read_packet);
    // ★ 这个连接是整个 UDP 收包链路的起点

    c.username = _ice_params.ice_ufrag;
    c.password = _ice_params.ice_pwd;
    ...
}

```

总结: `set_local_description` 不仅把 candidate 填入 SDP, 更创建了绑定在本地网卡上的 UDP socket, 并通过 `signal_read_packet` 连接把它变成了后续所有 ICE/DTLS/SRTP 数据的入口。

4. Candidate Pair → IceConnection: 从 UDP 四元组到逻辑通道

4.5 IceTransportChannel × Network → UDPPort 的数量关系

`IceTransportChannel` 是逻辑上的媒体传输通道, `UDPPort` 是附着在其上的物理 UDP socket。两者的数量关系受三层因素控制。

第一层 — BUNDLE 控制逻辑通道数 (`transport_controller.cpp:44`) :

```

if (desc->is_bundle(mid) && mid != desc->get_first_bundle_mid()) {
    continue; // audio + video 共享同一个 ICE channel
}

```

第二层 — RTCP mux 控制是否需要独立的 RTCP 通道: 创建时只传了

`IceCandidateComponent::RTP`。若不开启 RTCP mux, 则需额外创建 RTCP 通道。

第三层 — 网卡数量控制物理 socket 数: `gathering_candidate()` 遍历

`NetworkManager::get_networks()`, 为每个 Network 创建一个 `UDPPort`。

`NetworkManager` 有两种模式 (`src/base/network.cpp:19-50`) :

模式	触发条件	行为
云服务器手工配置	<code>conf.netcard</code> + <code>conf.ipv4_addr</code> 非空	跳过网卡扫描，手工构造唯一的 Network 对象
本地自动扫描	上述配置为空	<code>getifaddrs()</code> 遍历全部网卡，过滤 IPv4 非回环地址

云服务器必须手工配置的原因：公网 IP 在 NAT 环境下不绑定在任何本地网卡上，`getifaddrs()` 扫出来的是内网地址，客户端无法直连。

数量公式：

UDPPort 总数 = Network 数量 × IceTransportChannel 数量

场景	Network 数	IceTransportChannel 数	UDPPort 总数
云服务器（手工配公网 IP）+ BUNDLE + RTCP mux	1	1	1
本地 eth0 + BUNDLE + RTCP mux	1	1	1
本地 eth0 + wlan0 + BUNDLE + RTCP mux	2	1	2
本地 eth0 + wlan0 + BUNDLE off + RTCP mux	2	2	4
本地 eth0 + BUNDLE off + RTCP mux off	1	4	4

每个 UDPPort 对应一个独立的 UDP socket，绑定在对应网卡的 IP 上。服务端 SDP offer 列出所有 candidate，客户端逐一尝试，ICE 协议从多条 path 中选出最优。

层次全景：

```
set_local_description()
|
├─ IceAgent::create_channel() → IceTransportChannel（逻辑通道）
|   数量：BUNDLE + RTCP mux 决定
|
└─ IceAgent::gathering_candidate()
    |
    └─ for each channel:
        IceTransportChannel::gathering_candidate()
        |
        └─ for each Network (get_networks()):
            UDPPort::create_ice_candidate()
            │
            ├─ socket() + bind() + O_NONBLOCK
            ├─ AsyncUdpSocket 包裹
            ├─ signal_read_packet → _on_read_packet
            └─ Candidate 生成（携带 ice-ufrag + ice-pwd)
```

4.6 Candidate Pair → IceConnection: 从 UDP 四元组到逻辑通道

客户端也可能有多张网卡，每个网卡都可以向 SFU 的同一个 UDPPort 发送数据，形成多个 UDP 四元组。每个通过 STUN 校验的四元组对应一个 `IceConnection`。

UDP 四元组 = Candidate Pair:

```
客户端 WiFi:    192.168.1.100:50001  ┌
                                   │→ SFU UDPPort 120.76.197.143:10028
客户端 以太网:  10.0.0.50:50002    └

                               同一个 UDPPort
                               _connections map:
                               key "192.168.1.100:50001" → IceConnection #1
                               key "10.0.0.50:50002"    → IceConnection #2
```

`UDPPort` 内部用 `std::map<SocketAddress, IceConnection*> _connections` 按客户端地址索引 (`src/ice/udp_port.cpp:107-110`) :

```
IceConnection* UDPPort::get_connection(const rtc::SocketAddress& addr) {
    auto iter = _connections.find(addr);
    return iter == _connections.end() ? nullptr : iter->second;
}
```

当收到客户端 STUN Binding Request 且通过校验后，创建 `IceConnection` (`udp_port.cpp:327-342`) :

```
IceConnection* UDPPort::create_connection(const Candidate& remote_candidate) {
    IceConnection* conn = new IceConnection(_el, this, remote_candidate);
    auto ret = _connections.insert(
        std::make_pair(conn->remote_candidate().address, conn));
    if (ret.second == false && ret.first->second != conn) {
        ret.first->second = conn; // 同地址 → 替换旧连接
    }
    return conn;
}
```

层次结构:

```
UDPPort (物理层, 一个 UDP socket, 绑定一个本地地址)
├─ IceConnection #1 四元组 (client_wifi:50001, SFU:10028)
│   ├── write_state:  STATE_WRITABLE  ← ping 收到回复
│   ├── receiving:    true             ← 收到过对端数据
│   ├── rtt:          15ms
│   └── priority:      RFC 5245 计算值
├─ IceConnection #2 四元组 (client_eth:50002, SFU:10028)
│   ├── write_state:  STATE_WRITE_INIT ← 还没 ping 过
│   ├── receiving:    false
│   ├── rtt:          0
│   └── priority:      RFC 5245 计算值
└─ IceController —— 从 N 个连接中选出 1 个最优
```

```

└─ sort_and_switch_connection(): 5 级排序 + RTT 防抖(10ms)
    │   writable > write_state > receiving > priority > RTT
    │
└─ select_connection_to_ping(): 两层限速
    │   Channel 级 48ms/480ms, Connection 级 48ms/900ms/2500ms
    │
└─ _selected_connection → 唯一的发送出口

```

IceConnection 的职责——它是"最小逻辑通道", 承载候选对上的完整 ICE 状态:

职责	代码位置
STUN ping/pong	<code>ping()</code> / <code>on_connection_request_response()</code>
write state 生命周期	<code>set_write_state()</code> : INIT → WRITABLE → UNRELIABLE → TIMEOUT
receiving 方向探活	<code>update_receiving()</code> : 综合 ping + data 时间戳
RTT 平滑	<code>received_ping_response()</code> : $rtt = old * 0.75 + new * 0.25$
非 STUN 数据转发	<code>on_read_packet()</code> → <code>signal_read_packet</code> → DtlsTransport
数据发送	<code>send_packet(data, len)</code> → <code>_port->send_to(remote_addr)</code>

数据收发的唯一出口是 `_selected_connection` (`src/ice/ice_transport_channel.cpp:493-506`):

```

int IceTransportChannel::send_packet(const char* data, size_t len) {
    if (!_ice_controller->ready_to_send(_selected_connection)) return -1;
    return _selected_connection->send_packet(data, len);
}

```

四层职责总结:

层次	对象	职责
物理层	UDPPort	一个 UDP socket, 绑定本地地址, <code>_connections</code> map 管理所有四元组
逻辑层	IceConnection	每个候选对一个, 独立跟踪 write_state / receiving / RTT
决策层	IceController	5 级排序 + 两层限速, 选出最优连接
聚合层	IceTransportChannel	管理 UDPPort + IceController, 向外暴露 writable/receiving 信号

所有 IceConnection 共享同一个物理 socket, 但独立的状态决定了: 哪个连接被选中发数据、哪个连接已失效需要淘汰。DTLS 握手、SRTP 加解密、RTP 转发最终都通过**那一个** `_selected_connection` 的 `send_packet()` 发出, 通过 `on_read_packet()` → `signal_read_packet` 收进来。

5. ICE ping 定时器 + 两层限速 + 5 级排序

5.7 ICE ping 定时器的冷启动条件

`IceTransportChannel` 持有一个周期性定时器 (`_ping_watcher`)，周期在 48ms/480ms 之间自适应。冷启动的唯一门控是 `_maybe_start_pinging()` (`ice_transport_channel.cpp:414-428`)：

```
void IceTransportChannel::_maybe_start_pinging() {
    if (_start_pinging) return; // 已启动, 跳过

    if (_ice_controller->has_pingable_connection()) {
        _el->start_timer(_ping_watcher, _cur_ping_interval * 1000);
        _start_pinging = true;
    }
}
```

`has_pingable_connection()` → `_is_pingable()` 要求 `remote_candidate` 同时具备 username 和 password (`ice_controller.cpp:181-186`)：

```
if (remote.username.empty() || remote.password.empty()) {
    return false;
}
```

两把凭据的来源和时序：

凭据	何时填入	来源
<code>remote_candidate.username</code>	STUN Binding Request 解析成功	USERNAME 属性中提取的 <code>remote_ufrag</code>
<code>remote_candidate.password</code>	ANSWER 到达后补填	<code>set_remote_ice_params()</code> → <code>maybe_set_remote_ice_params()</code>

冷启动条件 = 至少一个 `IceConnection` 同时具备 ufrag (来自 STUN) 和 password (来自 ANSWER)。

`_maybe_start_pinging()` 在两个地方被调用，谁后到谁触发：

路径 A – STUN 先到，ANSWER 后到：

```
STUN Request → _on_unknown_address → _add_connection
→ _sort_connections_and_update_state → _maybe_start_pinging()
→ password 为空, has_pingable_connection() = false → 跳过
```

```
ANSWER 到 → set_remote_ice_params() → maybe_set_remote_ice_params()
→ 补填 password → _sort_connections_and_update_state
→ _maybe_start_pinging()
→ username+password 都到位 ★ 启动!
```

路径 B – ANSWER 先到，STUN 后到：

```
ANSWER → _remote_ice_params 写入, 此时还没有 connection → 跳过
```

```
STUN Request → _on_unknown_address
→ remote_candidate.password = _remote_ice_params.ice_pwd (已在 ANSWER 中填入)
→ 创建时密码就已经非空 → _add_connection
→ _sort_connections_and_update_state → _maybe_start_pinging() ★ 启动!
```

`_start_pinging` 保证只启动一次。

5.8 定时器的三职责 + 两层限速

`IceTransportChannel` 持有的定时器不止发 ping——它是整个 ICE 状态机的心跳驱动。

`_on_check_and_ping()` (`ice_transport_channel.cpp:448-465`) 每个周期做三件事：

```
void IceTransportChannel::_on_check_and_ping() {
    _update_connection_states(); // ① 巡检所有连接
    auto result = _ice_controller->select_connection_to_ping(...); // ② 选 ping
    目标
    if (result.conn) _ping_connection(conn); // ③ 发 ping
    // interval 变化 → 重启定时器
}
```

① `_update_connection_states()` ——遍历所有连接调 `conn->update_state(now)`，检测 ping 超时 → 降级 write_state (WRITABLE → UNRELIABLE → TIMEOUT)。状态变化时

`signal_state_change` → 触发 `_on_connection_state_change` →

`_sort_connections_and_update_state` → `sort_and_switch_connection()` 可能切换 selected connection。

② `select_connection_to_ping()` ——Channel 级 + Connection 级两层限速，选本轮应该 ping 的候选对。

③ `_ping_connection(conn)` ——对选中的候选对发出 STUN Binding Request，验证可达性。

每个定时器周期：

- ① 巡检所有连接 → 超时的降级/淘汰
- ② 连接状态变化 → 信号上报 → 可能触发 selected 切换
- ③ 选一个候选对发 ping → 验证可达性
- ④ 根据 weak/strong 调整定时器频率

以下详述步骤②中的两层限速设计。定时器周期是所有候选对的“最小分辨率”：通道稳定时加大周期省带宽，不稳定时缩小周期尽快收敛到最优。

第一层 — Channel 级 (`ice_controller.cpp:224-225`)：

```
int ping_interval = (_weak() || need_ping_more_at_weak)
    ? WEAK_PING_INTERVAL // 48ms
    : STRONG_PING_INTERVAL; // 480ms
```

`_weak()` = 没有 selected_connection，或者 selected 不满足 writable && receiving。

`need_ping_more_at_weak` = 存在 ping 未满 3 次的连接（新连接快速初探）。

第二层 — Connection 级 (`ice_controller.cpp:335-347`)：

条件	间隔	含义
< 3 次 ping	48ms	新连接，快速摸清质量
channel weak 或连接不稳定	900ms	中等频率
channel strong 且连接稳定	2500ms	低频保活

两层同时满足才真发包：

```
定时器 48ms 触发 → _on_check_and_ping()
→ select_connection_to_ping()
→ _find_next_pingable_connection()
→ _is_pingable 过滤:
    Channel 级: now >= last_ping_sent_ms + 48ms ← 定时器已保证
    Connection 级: now >= conn.last_ping_sent + 2500ms ← 稳定连接未到间隔
→ 跳过 → 本轮不发包
```

自适应调速器的本质:

定时器周期 = 所有候选对的"最小分辨率"

- |
- ├ weak 时 48ms: 高频探测, 每 48ms 至少 ping 一个 pair, 快速收敛到最优
- └ strong 时 480ms: 低频, 已经选出来了, 不需要频繁换

Connection 级在此基础上进一步限速:

weak 时: timer=48ms, _is_pingable 中 _weak() 直接返回 true, 不查 Connection 级间隔
strong 时: timer=480ms, 稳定连接 conn_interval=2500ms → 实际每 2500ms 才 ping 一次

关键: weak 时 Connection 级间隔被跳过。

_is_pingable() 中 _weak() 为 true 时直接 return true, 根本不走走到 _is_connection_past_ping_interval。所以 _get_connection_ping_interval 里的 900ms / 2500ms 在 weak 状态下完全不会被调用:

```
bool _is_pingable(IceConnection* conn, int64_t now) {
    if (remote.username.empty() || remote.password.empty()) return false;
    if (_weak()) return true; // ← 直接放行, 跳过 Connection 级限速
    return _is_connection_past_ping_interval(conn, now); // strong 时才进入
}
```

因此 48ms 和 900ms 从来不共存——它们在不同模式下各管各的:

weak 模式:

timer = 48ms
→ round-robin 公平轮转, 每 48ms 任一凭据齐全的连接都可能被选中 ping
→ 某个连接收到 response → writable → sort_and_switch → 成为 selected
→ selected 稳定 → _weak() 变 false → 切换 strong 模式

strong 模式:

timer = 480ms
→ selected 优先被 ping (2500ms 间隔)
→ 其他连接按各自间隔: 不稳定 900ms, 稳定 2500ms

这就是"快速收敛"的本质——48ms 高速轮询所有候选对, 直到有一个胜出, 此后降频省带宽。

已知问题: 当前采用 Aggressive Nomination——每包 ping 都带 USE-CANDIDATE

(stun_request.cpp:120), 包括轮询探活其他 pair 的 ping。这会导致客户端被探活 ping "误导"切到非 selected 的 pair。正确做法是 Regular Nomination: 只有 selected connection 的 ping 才带 USE-CANDIDATE。单网口场景下无实际影响 (仅 1 个 IceConnection), 多网卡时需修复。代码已标记 TODO。

5.13 ICE 5 级排序 + 两层限速：代码级走读

从第一个 UDP 包到达、创建 IceConnection、冷启动定时器，到稳态每周期探活、选连接、排序、升降档的完整代码链路。所有代码位置指向 `src/ice/`。

5.1 关键常量速查

常量	值	定义位置	含义
<code>WEAK_PING_INTERVAL</code>	48ms	<code>ice_def.h</code>	Channel weak 时的发包间隔 (10000bps 带宽假设)
<code>STRONG_PING_INTERVAL</code>	480ms	<code>ice_def.h</code>	Channel strong 时的发包间隔 (1000bps 带宽假设)
<code>STABLING_CONNECTION_PING_INTERVAL</code>	900ms	<code>ice_def.h</code>	单连接不稳定时的保活间隔
<code>STABLE_CONNECTION_PING_INTERVAL</code>	2500ms	<code>ice_def.h</code>	单连接稳定时的保活间隔
<code>MIN_PINGS_AT_WEAK_PING_INTERVAL</code>	3	<code>ice_def.h</code>	新连接快速初探的 ping 次数阈值
<code>k_min_improvement</code>	10ms	<code>ice_controller.cpp:11</code>	RTT 切换防抖: top 必须比当前 selected 至少小 10ms
<code>RTT_RATIO</code>	3	<code>ice_connection.cpp:19</code>	RTT 指数平滑权重: old : new = 3 : 1
<code>CONNECTION_WRITE_CONNECT_FAILS</code>	5	<code>ice_def.h</code>	连续未回复 ping 数 ≥5 → 触发降级检查
<code>CONNECTION_WRITE_CONNECT_TIMEOUT</code>	5000ms	<code>ice_def.h</code>	WRITABLE → UNRELIABLE 的超时阈值
<code>CONNECTION_WRITE_TIMEOUT</code>	15000ms	<code>ice_def.h</code>	UNRELIABLE/INIT → TIMEOUT 的超时阈值
<code>WEAK_CONNECTION_RECEIVE_TIMEOUT</code>	2500ms	<code>ice_def.h</code>	receiving 方向的超时
<code>PING_INTERVAL_DIFF</code>	5ms	<code>ice_transport_channel.cpp:13</code>	时钟容差，防定时器触发早于精确间隔导致错过周期

5.2 冷启动：从第一个 UDP 包到定时器启动

整个 ICE 探活系统的起点不是定时器，而是一个 UDP 包的到达。

第一步 — `_on_unknown_address` (`ice_transport_channel.cpp:138`)。

客户端发来 STUN Binding Request。 `UDPPort::_on_read_packet` 发现 `get_connection(addr)` 为 nullptr (未知地址)，走 `get_stun_message` → CRC32 fingerprint 校验通过 → 发射 `signal_unknown_address`。 `IceTransportChannel::_on_unknown_address` 接住后做五件事：

```
// 1. 从 STUN 消息中提取 PRIORITY 属性
const StunUInt32Attribute* priority_attr = stun_msg-
>get_uint32_t(STUN_ATTR_PRIORITY);

// 2. 构造 prflx candidate
Candidate remote_candidate;
remote_candidate.address = addr;           // 对端 IP:Port
remote_candidate.username = remote_ufrag; // 对端 ICE ufrag (从 STUN USERNAME 提
取)
remote_candidate.password = _remote_ice_params.ice_pwd; // △ 此时可能为空!
remote_candidate.type = PRFLX_PORT_TYPE;  // "prflx"

// 3. 创建 IceConnection
IceConnection* conn = port->create_connection(remote_candidate);
```

```
// 4. 注册到 controller + 连接三条信号线
_add_connection(conn);

// 5. 回复 STUN Binding Response (用 SFU 的 ice-pwd 做 MESSAGE-INTEGRITY)
conn->handle_stun_binding_request(stun_msg);

// 6. 首次排序 + 选路 + 可能启动定时器
_sort_connections_and_update_state();
```

密码的“先有鸡还是先有蛋”：第 2 步 `remote_candidate.password = _remote_ice_params.ice_pwd`——如果 ANSWER 还没到，`_remote_ice_params` 为空，密码就是空字符串。这会导致第 6 步 `_is_pingable` 返回 false，定时器暂不启动。后面客户端发来 ANSWER → `set_remote_ice_params` → `maybe_set_remote_ice_params` 补填密码 → 再次调 `_sort_connections_and_update_state` → 这次 `_is_pingable` 为 true → 定时器启动。

第二步 — `_add_connection` (`ice_transport_channel.cpp:195`)。连接三条信号线：

信号	触发时机	槽函数	做什么
<code>signal_state_change</code>	<code>set_write_state / update_receiving</code> 状态变化	<code>_on_connection_state_change</code>	重新排序 + 可能切换 selected
<code>signal_connection_destroy</code>	<code>IceConnection::destroy()</code>	<code>_on_connection_destroyed</code>	从 controller 清理；若销毁的是 selected → 重选
<code>signal_read_packet</code>	<code>on_read_packet</code> 判定非 STUN 包	<code>_on_read_packet</code>	透传给 DtlsTransport

同时在 controller 侧：`_connections.push_back(conn)` + `_unpinged_connections.insert(conn)` (`ice_controller.cpp:164`)。

第三步 — `_sort_connections_and_update_state` (`ice_transport_channel.cpp:360`)。编排三件事：

```
void IceTransportChannel::_sort_connections_and_update_state() {
    _maybe_switch_selected_connection(
        _ice_controller->sort_and_switch_connection()); // A. 排序 + 可能切换
    _update_state(); // B. 聚合
    writable/receiving
    _maybe_start_pinging(); // C. 可能首次启动定时器
}
```

A. `sort_and_switch_connection()` (`ice_controller.cpp:82`)。冷启动时 `_selected_connection == nullptr`，直接走到 `return top_connection`——“还没有 selected，排序后第一个就是”。`_switch_selected_connection` 将其设为 selected。

B. `_update_state()` (`ice_transport_channel.cpp:293`)：

```
bool writable = _selected_connection && _selected_connection->writable();
_set_writable(writable);

bool receiving = false;
for (auto conn : _ice_controller->connections()) {
    if (conn->receiving()) { receiving = true; break; } // 任意连接 receiving 就行
}
_set_receiving(receiving);
```

方向不对称： `writable` 只看 selected connection（数据发送走它），`receiving` 看任意连接（对端可能从多个路径发数据过来）。

C. `_maybe_start_pinging()` (`ice_transport_channel.cpp:414`)：

```
void IceTransportChannel::_maybe_start_pinging() {
    if (_start_pinging) return; // ★ 只启动一次，永不重复
    if (_ice_controller->has_pingable_connection()) {
        _el->start_timer(_ping_watcher, _cur_ping_interval * 1000); // 48ms
        _start_pinging = true;
    }
}
```

`has_pingable_connection()` (`ice_controller.cpp:145`) 遍历所有连接调 `_is_pingable`。只要有一个满足条件，定时器就启动。

冷启动时序小结：

```
UDP 收 STUN Binding Request
→ _on_unknown_address
→ create_connection (new IceConnection)
→ _add_connection (注册信号 + 加入 controller)
→ send_stun_binding_response (回复 Binding Response)
→ _sort_connections_and_update_state
→ sort_and_switch_connection (冷启动：无 selected → 直接选 top)
→ _update_state (writable=false, receiving=false)
→ _maybe_start_pinging (密码空 → has_pingable=false → 暂不启动)

... ANSWER 到达 ...

set_remote_ice_params
→ maybe_set_remote_ice_params (补填 _remote_candidate.password)
→ _sort_connections_and_update_state
→ _maybe_start_pinging (密码非空 → has_pingable=true → start_timer 48ms ★)
```

5.3 稳态： `_on_check_and_ping` 每周期循环

定时器启动后，libev 每 `_cur_ping_interval`（初始 48ms）回调 `ice_ping_cb` → `_on_check_and_ping` (`ice_transport_channel.cpp:448`)。五个步骤：

```
void IceTransportChannel::_on_check_and_ping() {
    _update_connection_states(); // ① 探活降级
    auto result = _ice_controller->select_connection_to_ping(
        _last_ping_sent_ms - PING_INTERVAL_DIFF); // ② 两层限速选连接
    if (result.conn) {
        _ping_connection(conn); // ③ 发出 STUN Binding Request
        _ice_controller->mark_connection_pingged(conn); // ④ unpinged → pinged
    }
    if (_cur_ping_interval != result.ping_interval) { // ⑤ 升降档
        _cur_ping_interval = result.ping_interval;
        _el->stop_timer(_ping_watcher);
        _el->start_timer(_ping_watcher, _cur_ping_interval * 1000);
    }
}
```

```
}
```

PING_INTERVAL_DIFF = 5ms **时钟容差**: `select_connection_to_ping` 的参数是 `_last_ping_sent_ms - 5`。假如定时器因系统调度在 479ms 触发而非精确 480ms, 用 `_last_ping_sent_ms - 5` 相当于把门控放宽 5ms, 避免因时钟粒度错过周期。

① `_update_connection_states` (`ice_transport_channel.cpp:474`)。遍历所有连接, 逐个调 `conn->update_state(now)`。降级逻辑见 §13.7。

② `select_connection_to_ping`: 核心调度逻辑, 见 §13.4 两层限速。

③ `_ping_connection` (`ice_transport_channel.cpp:488`):

```
void IceTransportChannel::_ping_connection(IceConnection* conn) {
    _last_ping_sent_ms = rtc::TimeMillis(); // ★ Channel 级全局时间戳
    conn->ping(_last_ping_sent_ms);
}
```

`conn->ping(now)` (`ice_connection.cpp:354`): 创建 `ConnectionRequest` (含 STUN Binding Request + 随机 `transaction_id`), 记录 `SentPing(id, now)` 到 `_pings_since_last_responses` 列表, 通过 `StunRequestManager::send()` 序列化并发出。 `_num_pings_sent++`。

④ `mark_connection_pinged` (`ice_controller.cpp:113`): 从 `_unpinged_connections` 移入 `_pinged_connections`。

⑤ **升降档**: `_weak()` 状态变化导致 `ping_interval` 在 48ms ↔ 480ms 之间切换时, 停止当前定时器、按新间隔重启。

5.4 第一层限速: Channel 级速率门

`select_connection_to_ping` (`ice_controller.cpp:214`) 先定 `ping_interval`, 再用它做门控。

```
PingResult IceController::select_connection_to_ping(int64_t last_ping_sent_ms) {
    // Step 1: 确定 channel 级 ping_interval
    bool need_ping_more_at_weak = false;
    for (auto conn : _connections) {
        if (conn->num_pings_sent() < MIN_PINGS_AT_WEAK_PING_INTERVAL) { // < 3
            need_ping_more_at_weak = true;
            break;
        }
    }
    int ping_interval = (_weak() || need_ping_more_at_weak)
        ? WEAK_PING_INTERVAL // 48ms
        : STRONG_PING_INTERVAL; // 480ms

    // Step 2: Channel 级门控
    int now = rtc::TimeMillis();
    const IceConnection* conn = nullptr;
    if (now >= last_ping_sent_ms + ping_interval) { // ★ 两用的
        last_ping_sent_ms
            conn = _find_next_pingable_connection(now); // 进入连接级选择
    }

    return PingResult(conn, ping_interval);
}
```

```
}
```

`ping_interval` 的三个决策分支：

条件	interval	场景
<code>_weak() = true</code>	48ms	selected 连接断了，急需探测新路径
存在 <code>num_pings_sent < 3</code> 的连接	48ms	新建连接快速完成初探（3 次 ping）
以上都不满足	480ms	已有稳定连接，低频保活即可

`_weak()` 定义（`ice_controller.h:54`）：

```
bool _weak() {
    return _selected_connection == nullptr
        || _selected_connection->weak();
}
// IceConnection::weak(): return !(writable() && receiving());
// 即"双向都通"才不算 weak
```

`need_ping_more_at_weak` 的作用：即使 selected connection 是 writable + receiving（不是 weak），只要还存在某个新连接未完成 3 次初探，channel 也维持 48ms 快速节奏，给所有连接公平的初探机会。

门控的含义：`last_ping_sent_ms` 是 channel 级全局时间戳——每次 `_ping_connection` 更新。整个 channel 所有连接共享一个发包时钟，一次定时器触发最多发一个 ping。

5.5 第二层限速：Connection 级间隔 + Round-Robin

Channel 级放行后，`_find_next_pingable_connection`（`ice_controller.cpp:252`）做连接级选择：

```
const IceConnection* IceController::_find_next_pingable_connection(int64_t now)
{
    // ① selected connection 优先
    if (_selected_connection && _selected_connection->writable() &&
        _is_connection_past_ping_interval(_selected_connection, now)) {
        return _selected_connection;
    }

    // ② unpinged 集合中还有可 ping 的吗？
    bool has_pingable = false;
    for (auto conn : _unpinged_connections) {
        if (_is_pingable(conn, now)) { has_pingable = true; break; }
    }

    // ③ unpinged 全部不可 ping → 一轮结束，全部倒回，开始新一轮
    if (!has_pingable) {
        _unpinged_connections.insert(_pinged_connections.begin(),
            _pinged_connections.end());
        _pinged_connections.clear();
    }
}
```

```

// ④ 在 unpinged 中选最 overdue 的连接
IceConnection* find_conn = nullptr;
for (auto conn : _unpinged_connections) {
    if (!_is_pingable(conn, now)) continue;
    if (_more_pingable(conn, find_conn)) {
        find_conn = conn; // last_ping_sent 更小 → 等得更久 → 优先
    }
}
return find_conn;
}

```

Round-Robin 双集合机制：

```

unpinged: {A, B, C} pinged: {}
周期1: ping A → unpinged: {B, C}, pinged: {A}
周期2: ping B → unpinged: {C}, pinged: {A, B}
周期3: ping C → unpinged: {}, pinged: {A, B, C}
周期4: unpinged 无可 ping → 倒回 → unpinged: {A, B, C}, pinged: {} ← 新一轮

```

为什么 selected connection 优先？ 它是数据发送的出口——所有 DTLS 握手包和 RTP 媒体包都经过它。优先 ping selected connection 保证数据通道的 writable 状态始终是最新鲜的。

`_more_pingable` (`ice_controller.cpp:293`)：比较 `last_ping_sent`，值更小的优先。等价于“谁等得最久就先 ping 谁”，保证 round-robin 公平。

`_is_pingable` 的三级判断 (`ice_controller.cpp:181`)：

```

bool IceController::_is_pingable(IceConnection* conn, int64_t now) {
    // ④ 对端凭据必须完整 (ANSWER 已到达)
    if (remote.username.empty() || remote.password.empty()) return false;

    // ④ channel weak → 跳过连接级间隔限制，任何有凭据的连接都可以 ping
    if (_weak()) return true;

    // ④ channel strong → 必须过连接级 ping 间隔
    return _is_connection_past_ping_interval(conn, now);
}

```

连接级间隔三级 (`_get_connection_ping_interval`, `ice_controller.cpp:335`)：

条件	间隔	语义
<code>num_pings_sent < 3</code>	48ms	新连接快速初探
<code>_weak()</code> 或 <code>!conn->stable(now)</code>	900ms	连接不稳定，中频探测
以上都不满足	2500ms	连接稳定，低频保活

`conn->stable()` 的定义 (`ice_connection.cpp:324`)：

```

bool IceConnection::stable(int64_t now) const {
    return _rtt_samples > RTT_RATIO + 1 // RTT 样本 > 4 (至少 5 次采样，平滑值可靠)
        && !_miss_response(now); // 没有 ping 等待超过 2*RTT (无丢包)
}

```

5.6 两层限速完整矩阵

Channel 级门		Connection 级门
控制整体发包节奏		保护单连接不被过度 ping
WEAK (48ms):	每 48ms 最多 1 个 ping true)	跳过 (_weak()) 直接返回
STRONG (480ms):	每 480ms 最多 1 个 ping / 稳定 2500ms	新连接 48ms / 不稳定 900ms

两层都通过才真正发包。举例：

场景	Channel 级	Connection 级	实际效果
新连接 + channel weak	48ms	跳过 (48ms 初探)	~48ms
新连接 + channel strong	480ms	48ms	实际受 channel 门限 ~480ms
稳定连接 + channel strong	480ms	2500ms	实际受 connection 门限 ~2500ms
连接断开 + channel weak	48ms	跳过	48ms 加速探测，尽快找到新路径

关键：Channel weak 时 connection 级门被绕过——此时 selected 连接已断，一切以最快找到新路径为优先，不需要保护连接。

5.7 连接写状态降级：update_state

`IceConnection::update_state()` (`ice_connection.cpp:244`) 是探活的核心，每次 `_on_check_and_ping` 循环第一步就调它。

RTT 容忍窗口：

```
int rtt = 2 * _rtt; // 2 倍当前 RTT 作为容忍窗口
if (rtt < MIN_RTT) rtt = MIN_RTT; // 钳位下限 100ms
else if (rtt > MAX_RTT) rtt = MAX_RTT; // 钳位上限 60s
```

两阶段退化：

阶段 1: `STATE_WRITABLE` → `STATE_WRITE_UNRELIABLE`

条件 (AND):

`_too_many_ping_failed(5, rtt, now)` → 第 5 个未回复 ping 已过 rtt 容忍窗口
`_too_long_without_response(5000, now)` → 最早未回复 ping 等待 >5s

阶段 2: `STATE_WRITE_UNRELIABLE` / `STATE_WRITE_INIT` → `STATE_WRITE_TIMEOUT`

条件:

`_too_long_without_response(15000, now)` → 最早未回复 ping 等待 >15s

`_too_many_ping_failed` 的逻辑 (`ice_connection.cpp:211`) :

```
bool IceConnection::_too_many_ping_failed(size_t max_pings, int rtt, int64_t now) {
    if (_pings_since_last_responses.size() < max_pings) return false;
    // 取第 max_pings 个 (0-indexed = 第 5 个) 未回复 ping
    int expected_response_time = _pings_since_last_responses[max_pings - 1].sent_time + rtt;
    return now > expected_response_time;
    // "第 5 个 ping 发出去后, 过了 2*RTT 还没收到任何回复" → 连续失败 ≥5 次
}
```

为什么阶段 1 需要两个条件同时成立? 单次 ping 丢包是正常的 (UDP 不可靠), 必须连续 5 次失败 + 超过 5 秒才确认"不是偶然丢包、确实出了问题"。两个条件互补——RTT 容忍窗口适应快速变化的网络, 5 秒硬超时兜底极端情况。

`update_receiving` 末尾调用 (`ice_connection.cpp:414`) :

```
void IceConnection::update_receiving(int64_t now) {
    bool receiving = false;
    if (_last_ping_sent < _last_ping_response_received) {
        receiving = true; // 发了 ping, 之后收到了对方的 ping response → 对端活着
    } else {
        receiving = last_received() > 0
            && (now < last_received() + receiving_timeout()); // 2500ms 窗口内收到过数据
    }
    if (_receiving != receiving) { _receiving = receiving;
    signal_state_change(this); }
}
```

第一条判断路径 `_last_ping_sent < _last_ping_response_received`: 我们发的 ping 是对端用 `_remote_candidate.password` (客户端密码) 验证的, 客户端回复的 Binding Response 也是用同一个密码做 MI, 说明客户端确实知道自己的密码——对端身份确认、路径存活。

5.8 5 级排序算法 + RTT 防抖

`sort_and_switch_connection` (`ice_controller.cpp:82`) 被

`_sort_connections_and_update_state` 调用——每次连接状态变化时触发, 不仅仅是定时器周期。

```
IceConnection* IceController::sort_and_switch_connection() {
    // Step 1: stable_sort - 多级比较器 + RTT fallback
    absl::c_stable_sort(_connections, [this](IceConnection* a, IceConnection* b)
    {
        int cmp = _compare_connections(a, b);
        if (cmp != 0) return cmp > 0; // 前 5 级有明确胜负
        return a->rtt() < b->rtt(); // 第 6 级: RTT fallback
    });

    IceConnection* top = _connections.empty() ? nullptr : _connections[0];

    // Step 2: 三种不切换的情况
    if (!ready_to_send(top) || top == _selected_connection) return nullptr;
```

```
// Step 3: 冷启动 - 还没有 selected, 直接选
if (!_selected_connection) return top;

// Step 4: RTT 防抖 - top 的 RTT 必须比当前 selected 至少小 10ms
if (top->rtt() <= _selected_connection->rtt() - k_min_improvement) {
    return top; // 值得切换
}
return nullptr; // RTT 差距不够 -> 不切换, 防止 ping-pong 振荡
}
```

为什么用 `stable_sort`? 保证排序稳定——前 5 级相等 + RTT 相等的连接保持原有相对顺序, 避免每次排序都改变 selected connection, 造成不必要的切换。

`_compare_connections` **5 级比较器** (`ice_controller.cpp:28`) :

级别	比较项	规则	为什么这一级不可或缺
1	<code>writable()</code>	true > false	能发数据是首要条件。writable=false 直接淘汰
2	<code>write_state()</code>	值小的 > 值大的	writable 是二值, 两个 writable=true 的连接可能一个 WRITABLE(0)、一个 UNRELIABLE(1)——需要更细粒度区分
3	<code>receiving()</code>	true > false	对端→我方向还活着。两个都 writable 但一个对端已失活的数据通道不可靠
4	<code>priority()</code>	值大的 > 值小的	RFC 5245 公式: <code>2^32*min(G,D) + 2*max(G,D) + (G>D?1:0)</code> , 编码了 candidate 类型偏好
5	<code>rtt()</code>	值小的 > 值大的	<code>stable_sort</code> lambda 中 fallback。前 4 级全相等时, RTT 最小的最优先

为什么 `write_state` 单独一级而不是只看 `writable`? 假设两个连接:

```
Conn A: writable=true, write_state=STATE_WRITE_UNRELIABLE (1)  ← 不稳定但还能用
Conn B: writable=true, write_state=STATE_WRITABLE (0)           ← 健康
```

如果只看 writable, 两者打平, 得靠后面的优先级和 RTT 决定——可能不稳定连接反而胜出。有了 write_state 这一级, Conn B 直接胜出, 不必走到后面的比较。

`ready_to_send` (`ice_controller.cpp:64`) :

```
bool IceController::ready_to_send(IceConnection* conn) {
    return conn && (conn->writable() ||
        conn->write_state() == IceConnection::STATE_WRITE_UNRELIABLE);
}
```

`STATE_WRITE_UNRELIABLE` 也能发——“不太稳定但还没死，数据照样发，同时继续 ping 找更好的”。只有 `STATE_WRITE_TIMEOUT` 才真正不能发。

RTT 防抖的经济学解释：两连接 RTT 分别是 15ms 和 20ms，差距仅 5ms。如果无阈值就切换，可能下一个 ping 周期 RTT 变成 17ms vs 18ms 又切回去——切换本身有代价（selected 变更可能触发上层信号链）。10ms 阈值把这种微小的随机波动过滤掉，只在“新连接明显更好”时才切。

5.9 RTT 指数平滑

`received_ping_response` (`ice_connection.cpp:462`) :

```
void IceConnection::received_ping_response(int rtt) {
    if (_rtt_samples > 0) {
        _rtt = rtc::GetNextMovingAverage(_rtt, rtt, RTT_RATIO);
        // 等价于: _rtt = (_rtt * 3 + rtt) / 4 → 旧值 75% + 新值 25%
    } else {
        _rtt = rtt; // 首次测量直接赋值，不参与平滑（避免初始 outlier 锚定）
    }
    ++_rtt_samples;
    _last_ping_response_received = rtc::TimeMillis();
    _pings_since_last_responses.clear(); // 收到回复 → 清空未回复 ping 列表
    update_receiving(_last_ping_response_received);
    set_write_state(STATE_WRITABLE);
    set_state(IceCandidatePairState::SUCCEEDED);
}
```

权重 3:1 的含义：RTT 对单次网络抖动敏感，但平滑后不能太迟钝。3:1 是 RFC 6298 TCP RTO 的经典权重——足够平滑以过滤噪声，又足够灵敏以反映趋势变化。

首次测量直接赋值：如果第一次 RTT 测量值是异常的 500ms（偶然丢包重传），用平滑公式的话初始 `_rtt` 被锚定在 500ms，后续测量 10ms 也只能缓慢下拉。直接赋值让初始值无偏，随后平滑自然收敛到真实值。

5.10 write_state 超时 → k_failed → UAF 防护链

当一个连接的 `write_state` 降级到 `STATE_WRITE_TIMEOUT` → `active() = false` → `_compute_ice_transport_state()` (`ice_transport_channel.cpp:324`) 检测到“曾经有连接，现在全部 inactive” → 返回 `k_failed`：

```
IceTransportState IceTransportChannel::_compute_ice_transport_state() {
    bool has_connection = false;
    for (auto conn : connections()) {
        if (conn->active()) { has_connection = true; break; }
    }
    if (_had_connection && !has_connection) return IceTransportState::k_failed;
    // ★
    ...
}
```

`k_failed` → `signal_ice_state_change` → `IceAgent::_update_state` → `signal_ice_state` → `TransportController::_on_ice_state` → `_update_state` → `k_failed` → `signal_connection_state` → `PeerConnection` → `RtcStream` → `on_connection_state(k_failed)` → `_remove_push_stream` → `delete push_stream` →

`~RtcStream` → `_pc->destroy()` (10ms 延迟定时器)。

这正是 §12.4 讲的 UAF 防护场景。整个析构链从 `_on_check_and_ping` →

`_update_connection_states` → `conn->update_state` 的调用栈内触发。

`PeerConnection::destroy()` 的 10ms 延迟保证 `_on_check_and_ping` 返回后不会访问已析构的 `_ice_controller`。

5.11 状态变化驱动的重排序路径

非定时器驱动的重排序由两条路径触发：

路径 A — `_on_connection_state_change`：

```
set_write_state / update_receiving → signal_state_change
  → _on_connection_state_change
    → _sort_connections_and_update_state
      → sort_and_switch_connection (RTT 防抖切换)
      → _update_state
      → _maybe_start_pinging (已启动 → 跳过)
```

路径 B — `_on_connection_destroyed`：

```
IceConnection::destroy() → signal_connection_destroy
  → _on_connection_destroyed
    → 从 controller 清理 (connections + pinged + unpinged)
    → 如果销毁的是 selected → _switch_selected_connection(nullptr) + 重排序
    → _update_state (receiving 可能重算——销毁非 selected 连接后仍需重新判断)
```

路径 B 的 `_update_state` 很重要：`receiving` 看任意连接，销毁一个连接后可能从 `receiving=true` 变为 `false`，必须重新计算。

5.12 完整调用链总览

== 冷启动 ==

```
UDP 收 STUN Binding Request
  → UDPPort::_on_read_packet
    → signal_unknown_address
      → IceTransportChannel::_on_unknown_address
        → port->create_connection(remote_candidate) // new IceConnection
        → _add_connection(conn) // 注册信号 + controller
        → conn->handle_stun_binding_request(stun_msg) // 回复 Binding
Response
  → _sort_connections_and_update_state
    → sort_and_switch_connection() // 冷启动：无 selected →
直接选
  → _update_state() // writable + receiving
聚合
  → _maybe_start_pinging() // 密码空 → 暂不启动

ANSWER 到达
  → set_remote_ice_params
  → maybe_set_remote_ice_params (补填 password)
```

```

→ _sort_connections_and_update_state
→ _maybe_start_pinging() // 密码非空 →
start_timer(48ms) ★

```

== 稳态循环 ==

```

libev timer (48ms / 480ms)
→ ice_ping_cb
→ _on_check_and_ping
  └ _update_connection_states
    └ for each conn: conn->update_state(now)
      └ WRITABLE → UNRELIABLE? (>=5 次失败 && >5s)
      └ UNRELIABLE/INIT → TIMEOUT? (>15s 无回复)
      └ update_receiving(now)
    |
  └ select_connection_to_ping(last_ping - 5ms)
    └ 定 ping_interval: _weak()? 或 need_ping_more? → 48ms : 480ms
    └ Channel 门: now >= last_ping_sent_ms + ping_interval?
    └ _find_next_pingable_connection(now)
      └ selected 优先? (writable && 过连接级间隔)
      └ unpinged 无可 ping? → pinged 全部倒回 unpinged
      └ _is_pingable 过滤 (凭据 + weak/间隔)
      └ _more_pingable 选最 overdue (last_ping_sent 最小)
    |
  └ _ping_connection(conn)
    └ _last_ping_sent_ms = now ← Channel 级时间戳
    └ conn->ping(now)
      └ new ConnectionRequest (STUN Binding + transaction_id)
      └ _pings_since_last_responses.push_back(SentPing)
      └ _request_manager.send() → UDP socket
      └ _num_pings_sent++
    |
  └ mark_connection_pinged(conn) ← unpinged → pinged
  |
  └ 升降档: interval 变了? → 重启定时器

```

== 收到 STUN Binding Response ==

```

UDP 收包
→ IceConnection::on_read_packet
→ STUN_BINDING_RESPONSE case
→ validate_message_integrity(_remote_candidate.password)
→ _request_manager.check_response() → 按 transaction_id 匹配
→ ConnectionRequest::on_request_response
→ on_connection_request_response
→ received_ping_response(rtt)
  └ RTT 指数平滑 (old*0.75 + new*0.25)
  └ _pings_since_last_responses.clear()
  └ update_receiving → signal_state_change
  └ set_write_state(STATE_WRITABLE) → signal_state_change
  └ set_state(SUCCEEDED)

```

== 状态变化触发重排序 ==

signal_state_change (由 set_write_state / update_receiving 发射)

```

→ _on_connection_state_change
→ _sort_connections_and_update_state
→ sort_and_switch_connection()
    |─ stable_sort (5级比较 + RTT fallback)
    |─ top 不是 writable/UNRELIABLE 或 top == selected → 不切
    |─ 无 selected → 冷启动, 直接选
    |─ top RTT <= selected RTT - 10ms → 切换
→ _update_state()
→ _maybe_start_pinging() (已启动 → 跳过)

```

6. ICE 四层状态机

6.9 ICE 状态机：从 IceConnection 到 IceTransportChannel 到 IceAgent

ICE 状态机分四层，每层聚合上一层的信息。

第一层：IceConnection —— 单个候选对的三维状态

一个 `IceConnection` 代表一个 UDP 四元组（候选对），跟踪三个独立维度：

Candidate Pair State (RFC 5245)：

```

WAITING → IN_PROGRESS (发 ping) → SUCCEEDED (收到回复)
        → FAILED → destroy()

```

实际用处不大，更多是 RFC 合规。真正驱动上层行为的是下面两个。

Write State —— “我们→对端”方向：

```

INIT(2) ——收到ping response——→ WRITABLE(0)
          ——5个连续超时 + 等5s——→ UNRELIABLE(1)
          ——等15s——→ TIMEOUT(3)

```

升级在 `received_ping_response()` 中（收到 ping 响应直接跳到 WRITABLE），退化在 `update_state(now)` 中（`ice_connection.cpp:244-273`）：

退化阶段	条件	阈值
WRITABLE → UNRELIABLE	>=5 个连续 ping 超时 且 最早未回复等了 5s	<code>CONNECTION_WRITE_CONNECT_FAILS=5</code> , <code>CONNECTION_WRITE_CONNECT_TIMEOUT=5000</code>
UNRELIABLE/INIT → TIMEOUT	最早未回复等了 15s	<code>CONNECTION_WRITE_TIMEOUT=15000</code>

双条件设计：“5 个连续超时”防止偶发丢包误判，“等 5s”防止大 RTT 时快速误判。RTT 容忍窗口 = $2 * _rtt$ ，钳位在 `[100ms, 60000ms]`。

Receiving —— “对端→我们”方向（`update_receiving()`，`ice_connection.cpp:414-430`）：

路径	条件	含义
1	<code>_last_ping_sent < _last_ping_response_received</code>	我发了 ping 之后收到了 response ——对端在听
2	<code>last_received() > 0 && now < last_received() + 2500ms</code>	最近 2.5s 收到过任何数据

`last_received()` = max(最后一次收 ping, 最后一次收 ping response, 最后一次收数据), 三个时间戳按需更新。**writable 和 receiving 独立判断**——UDP 链路可以单向通。

RTT 指数平滑 + 稳定判断:

```
// 首次: rtt = 测量值 (不参与平滑, 避免混入默认值3000)
// 后续: rtt = old*0.75 + new*0.25
```

`stable() = _rtt_samples > 4 && !_miss_response(now)`。至少 5 个样本 + 无等待超过 $2 * rtt$ 的 ping。

weak() 的定义:

```
bool weak() { return !(writable() && receiving()); }
```

双向都通才叫 strong。这个定义直接影响 Controller 的 ping 频率选择。

第二层: IceTransportChannel —— 聚合所有连接

writable = selected one:

```
bool writable = _selected_connection && _selected_connection->writable();
```

只看 selected connection。没有 selected → writable = false。

receiving = any one:

```
bool receiving = false;
for (auto conn : connections()) {
    if (conn->receiving()) { receiving = true; break; }
}
```

与 writable 不对称——一个要求"选中的能发", 一个要求"任意一个在收"。

两个单调标记:

标记	置 true 时机	含义
<code>_had_connection</code>	<code>_add_connection()</code> 调用时	曾经创建过连接
<code>_has_been_connection</code>	<code>_set_writable(true)</code> 调用时	曾经 writable 过

永不回退。用于区分不同失败类型。

七个状态的优先级计算 (`_compute_ice_transport_state()`, `ice_transport_channel.cpp:324-350`):

① <code>_had_connection && !has_connection</code>	<code>→ k_failed</code>	"曾有连接，全死"
② <code>_has_been_connection && !writable</code>	<code>→ k_disconnected</code>	"曾经通，现在断"
③ <code>!_had_connection && !has_connection</code>	<code>→ k_new</code>	"从未有连接"
④ <code>has_connection && !writable</code>	<code>→ k_checking</code>	"有候选对，在 ping"
⑤ 以上都不是	<code>→ k_connected</code>	"一切正常"

`k_failed` vs `k_disconnected` 的区别：

`k_failed`: 所有连接 `TIMEOUT` `→` 候选对全死 `→` 不可恢复
`k_disconnected`: `selected` 不通，但候选对还在 `→` 可能切备用恢复

`k_failed` 最终触发 `RtcStreamManager` 销毁流 (`rtc_stream_manager.cpp:181-191`)。销毁链路见 §12。

第三层：IceAgent —— 聚合多个 Channel

BUNDLE 下通常只有一个 Channel。多个 Channel 时逐状态计数，按优先级聚合 (`ice_agent.cpp:76-118`)：

规则：failed > disconnected > new > checking > completed > connected

特殊处理：`k_checking` `→` `k_completed` 转换时人为注入 `k_connected` 信号，确保上层不跳过 `k_connected` (否则 30s 超时定时器不会被删除)。

第四层：TransportController —— ICE + DTLS 聚合为 PC 状态

`TransportController::_update_state()` 同时取 `DtlsTransport::dtls_state()` 和 `IceTransportChannel::state()`，每个 transport 算 dtls + ice 两份 (`transport_controller.cpp:128-171`)：

规则：any failed `→` `k_failed`
 any ice_disconnected `→` `k_disconnected`
 all new+closed `→` `k_new`
 any checking/connecting `→` `k_connecting`
 all connected/completed/closed `→` `k_connected`

注意：DTLS 侧的 `dtls_state()` 是否变化会影响 PC 状态——DTLS 握手失败同样可以触发 `k_failed`。DTLS 状态机待后续深挖。

状态变化传播链：

```

每个定时器周期 _on_check_and_ping():
    → _update_connection_states() → conn->update_state()
    → write_state 退化 → signal_state_change
    → _sort_connections_and_update_state()
    → _update_state()
    → _set_writable → _has_been_connection = true →
signal_writable_state_change
    → _set_receiving → signal_receiving_state_change
    → _compute_ice_transport_state → signal_ice_state_change
    → IceAgent._update_state → signal_ice_state
    → TransportController._update_state → signal_connection_state
    → RtcStream._on_connection_state
    → k_connected: 删除 30s 超时定时器
    → k_failed: _listener->on_connection_state → delete stream

```

这就是为什么 `PeerConnection::destroy()` 需要 10ms 延迟析构——整条信号链的起点在 `_on_check_and_ping()` 的调用栈内。若同步 `delete pc`, `_ice_controller` 被析构后调用方还要访问它 → `coredump`。

已知问题: receiving 信号链完整但上层未使用。

```

IceConnection::_receiving ← 唯一被实际使用的层 (Controller._weak())
→ IceTransportChannel::_set_receiving() ← 存储, 无人查询
→ DtlsTransport::_set_receiving() ← 存储, 无人查询
→ TransportController::_on_dtls_receiving_state → _update_state() ← 不看值

```

四层中只有 `IceController::_weak()` 直接读 `IceConnection::receiving()`。上层三层的 receiving 存储和信号转发仅作为 `_update_state()` 的触发源, 但 `_update_state()` 内部不看 receiving bool, 只看 `IceTransportState` / `DtlsTransportState` 枚举值。如果将来需要在 PC 状态判断中考虑 receiving (比如 receiving=false 时即使 writable 也降级为 k_disconnected), 骨架已就位。

7. SDP 字段 → ICE/DTLS 的"最后一公里"

7.1 字段提取

入口 `PeerConnection::set_remote_sdp()` (`src/pc/peer_connection.cpp:344`) :

```

"a=ice-ufrag:AbCd" → TransportDescription::ice_ufrag = "AbCd"
"a=ice-pwd:xxxxxx" → TransportDescription::ice_pwd = "xxxxxx"
"a=fingerprint:sha-256 AB:CD:..." → TransportDescription::identity_fingerprint

```

解析函数 `parse_transport_info()` (`peer_connection.cpp:171-207`) 用 `starts_with` 匹配行前缀。提取完存入 `SessionDescription::_transport_infos`。

7.2 分发: 同一个函数同时喂 ICE 和 DTLS

`TransportController::set_remote_description()` (`src/pc/transport_controller.cpp:219-245`) :

```

auto td = remote_desc->get_transport_info(mid);
if (td) {

```

```
// BRANCH A: ice-ufrag + ice-pwd → ICE 子系统
_ice_agent->set_remote_ice_params(mid, IceCandidateComponent::RTP,
    IceParameters(td->ice_ufrag, td->ice_pwd));

// BRANCH B: fingerprint → DTLS 子系统
DtlsTransport* dtls = _get_dtls_transport(mid);
if (dtls) {
    dtls->set_remote_fingerprint(td->identity_fingerprint->algorithm,
        td->identity_fingerprint->digest.data(),
        td->identity_fingerprint->digest.size());
}
}
```

注意顺序：先设 ICE 参数，再设 DTLS 指纹。`set_remote_ice_params` 会触发 `_sort_connections_and_update_state()`，可能首次满足 ping 条件并启动 ICE 连通性检查。

7.3 ice-ufrag/pwd 的两把密码 + 各自消费点

ICE 涉及**两把**密码，对称分工。规则见 §8.1：“发给谁，就用谁的密码”。

本地密码 `_ice_params.ice_pwd`（SFU 密码）：

消费点	场景	代码位置
验证客户端发来的 Binding Request	收到	<code>udp_port.cpp:195</code> — <code>validate_message_integrity(_ice_params.ice_pwd)</code>
构造发给客户端的 Binding Response	发出	<code>ice_connection.cpp:62</code> — <code>add_message_integrity(_port->ice_pwd())</code>

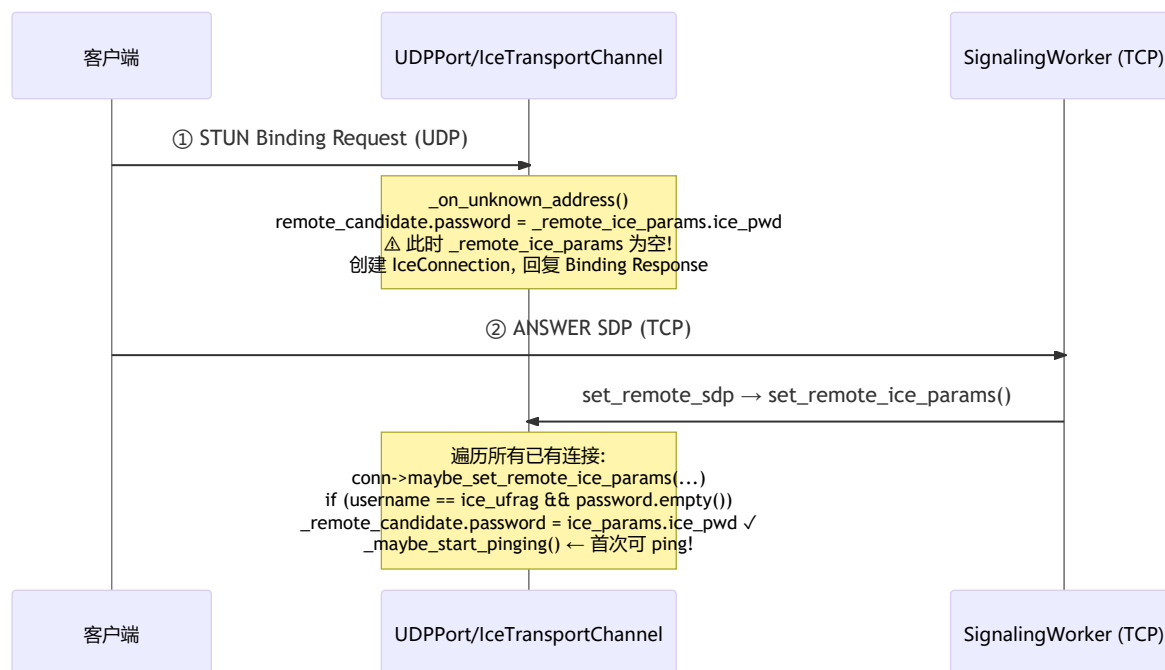
远端密码 `_remote_ice_params.ice_pwd`（客户端密码）：

进入 `IceTransportChannel` 后存在 `_remote_ice_params` 成员（`ice_transport_channel.cpp:75`），随后通过 `maybe_set_remote_ice_params` 补填到已有连接的 `remote_candidate.password`（`ice_connection.cpp:309-315`）。消费点：

消费点	场景	代码位置
分配给 <code>remote_candidate.password</code>	创建 prflx 时赋值	<code>ice_transport_channel.cpp:162</code> — 此时 ANSWER 可能未到，为空，需 §3.4 补填
ping 门控	判断 <code>_is_pingable()</code>	<code>ice_controller.cpp:183</code> — <code>remote.password</code> 非空 = ANSWER 已到
构造发给客户端的 Binding Request (ping)	发出	<code>stun_request.cpp:131</code> — <code>add_message_integrity(remote_candidate().password)</code>
验证客户端发来的 Binding Response	收到	<code>ice_connection.cpp:124</code> — <code>validate_message_integrity(_remote_candidate.password)</code>

注意：之前说远端密码“不是 HMAC key，只是门控标志位”——这是错的。远端密码既做门控（`_is_pingable`），也做 HMAC key（ping 时构造 MI、收 response 时验证 MI）。“发给谁就用谁的密码”——发给客户端时用客户端密码，正是远端密码。

7.4 密码的"先有鸡还是先有蛋"问题



代码位置 `src/ice/ice_connection.cpp:309-315` :

```
void IceConnection::maybe_set_remote_ice_params(const IceParameters& ice_params)
{
    if (_remote_candidate.username == ice_params.ice_ufrag &&
        _remote_candidate.password.empty()) {
        _remote_candidate.password = ice_params.ice_pwd;
    }
}
```

7.5 fingerprint 的两个时序路径

`DtlsTransport::set_remote_fingerprint()` (`src/pc/dtls_transport.cpp:448-500`) :

路径 A (正常: ANSWER 先到) :

```
set_remote_fingerprint -> 存 _remote_fingerprint_alg/value
-> _dtls 还不存在 -> 跳过两个 if
-> _setup_dtls() -> SetPeerCertificateDigest(已存的指纹) -> 一把配好
```

路径 B (乱序: ClientHello 先到) :

ClientHello 触发 `_setup_dtls()` → 当时指纹空, `SetPeerCertificateDigest` 被跳过
→ ANSWER 后到 → `_dtls` 已存在 && `!is_fingerprint_change`
→ 直接对已创建的 `_dtls` 补调 `SetPeerCertificateDigest` (line 475-482)

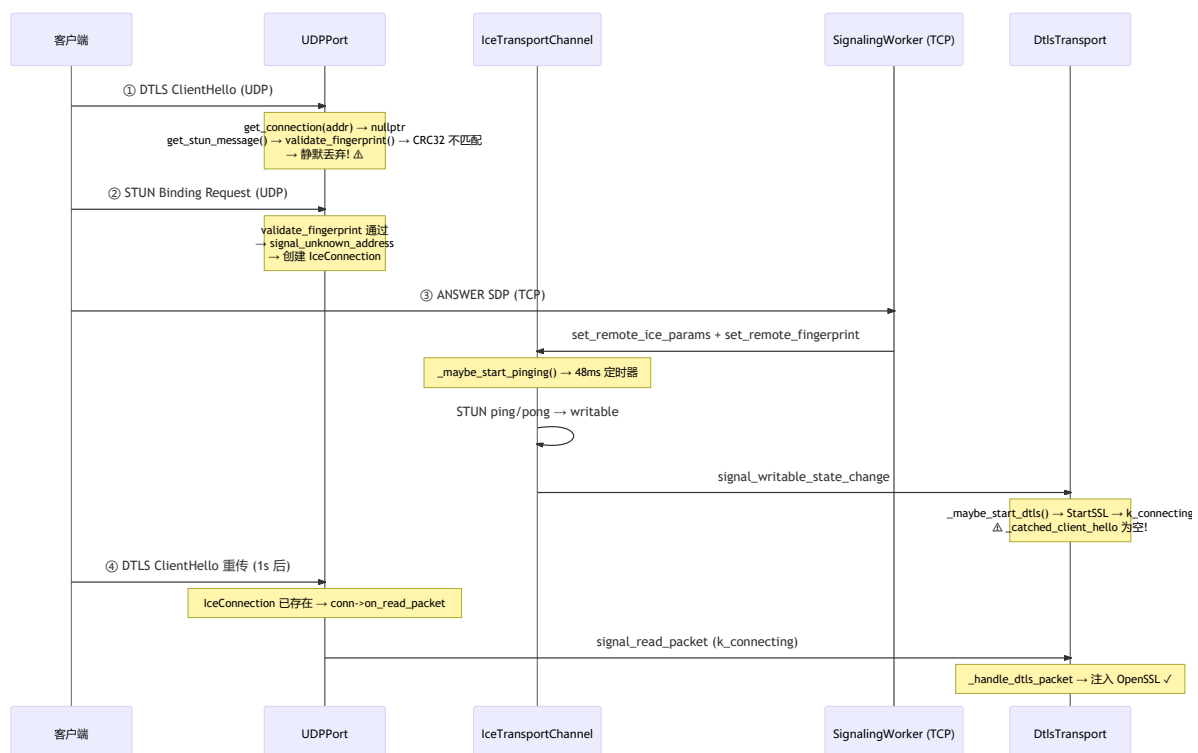
8. 三包竞态：STUN / DTLS ClientHello / ANSWER 时序全集

STUN Binding Request (UDP)、DTLS ClientHello (UDP)、ANSWER SDP (TCP) 三者从客户端几乎同时发出，但在服务端以任意顺序到达。

8.1 六种时序

#	到达顺序	发生什么	延迟影响
1	STUN → DTLS → ANSWER	S 创建连接 → D 缓存到 <code>_cached_client_hello</code> → A 补指纹+密码 → ICE writable → 重放 D	无延迟
2	STUN → ANSWER → DTLS	S 创建连接 → A 补全凭据+指纹 → D 到达时 DTLS 已配好指纹	无延迟
3	ANSWER → STUN → DTLS	A 先到 (UDP 上无连接) → S 创建连接+立即可 ping → ICE writable → D 到达	无延迟
4	DTLS → STUN → ANSWER	D 先到 无连接 → <code>validate_fingerprint</code> 失败 → UDPPort 静默丢弃 → S 创建 → A 补凭据 → 客户端 重传 D (1s)	+1s DTLS 重传
5	DTLS → ANSWER → STUN	D 被丢弃 → A 到达 (无连接) → S 创建 → 等重传	+1s
6	ANSWER → DTLS → STUN	A 到达 (无连接) → D 被丢弃 → S 创建 → 等重传	+1s

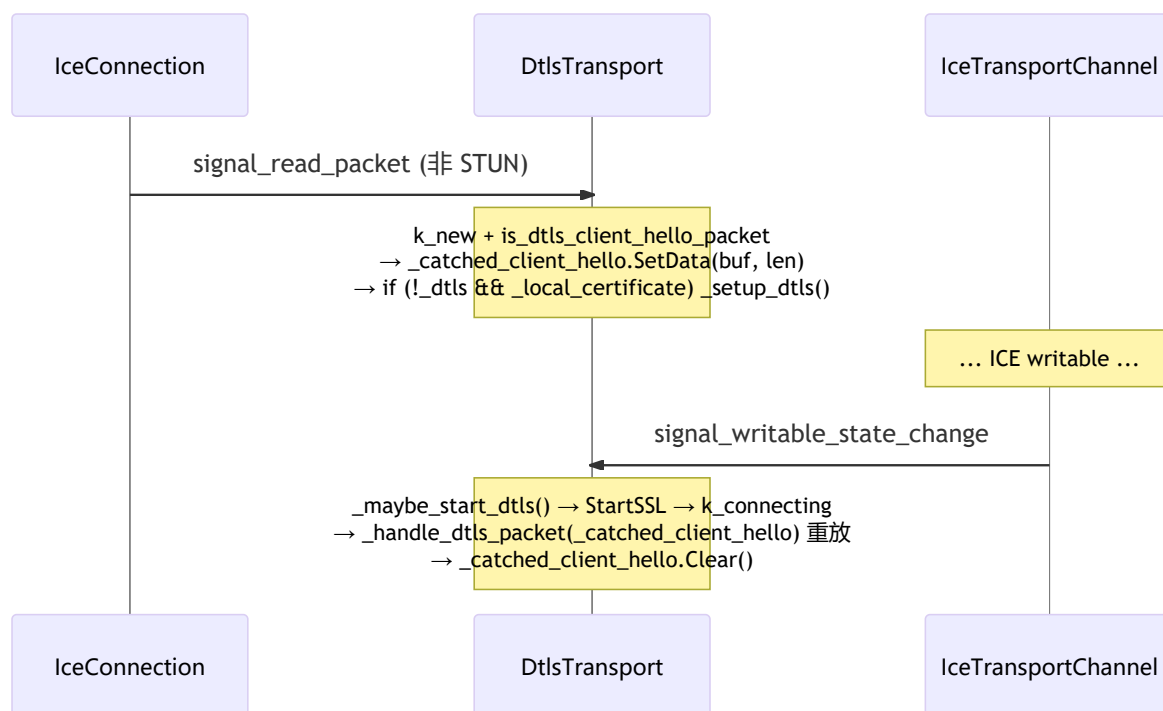
8.2 时序 4 详细走读 (唯一有延迟损失的路径)



设计取舍：UDPPort 不可能为每个未知 UDP 包都缓存（DoS 向量），所以选择"丢弃 + 依赖 DTLS 重传"。+1s 是 RFC 6347 规定的 DTLS 初始重传超时。

8.3 `_cached_client_hello` 的真实作用范围

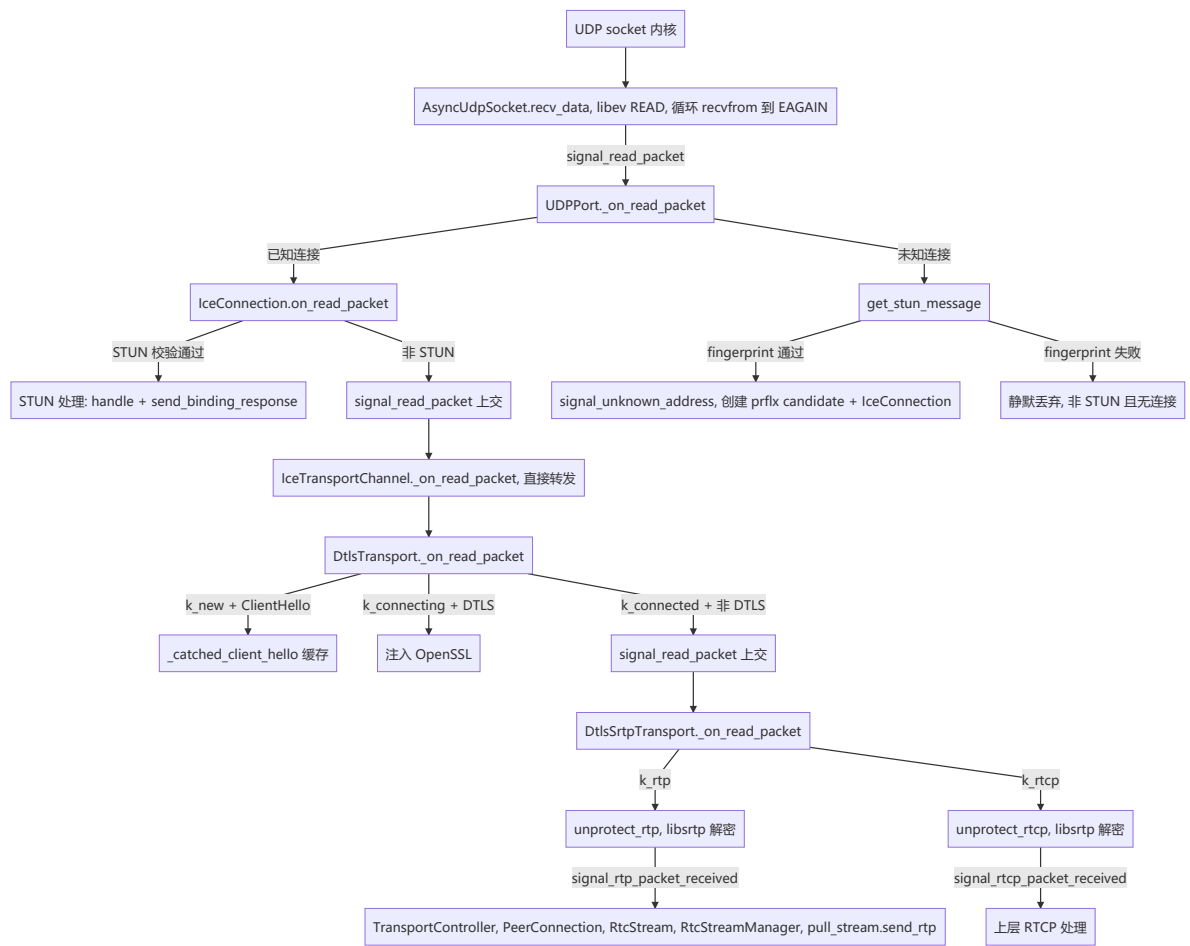
只处理 DTLS ClientHello 在 STUN 之后但在 DTLS 启动之前到达的情况（时序 1），不处理 D 先于 STUN 到达的情况。



9. 同一个 UDP socket 如何服务三层协议

同一个 UDP socket、同一个端口，收包走同一条路径，然后在四个分拣层逐级分流。

9.1 收包分拣链（完整）

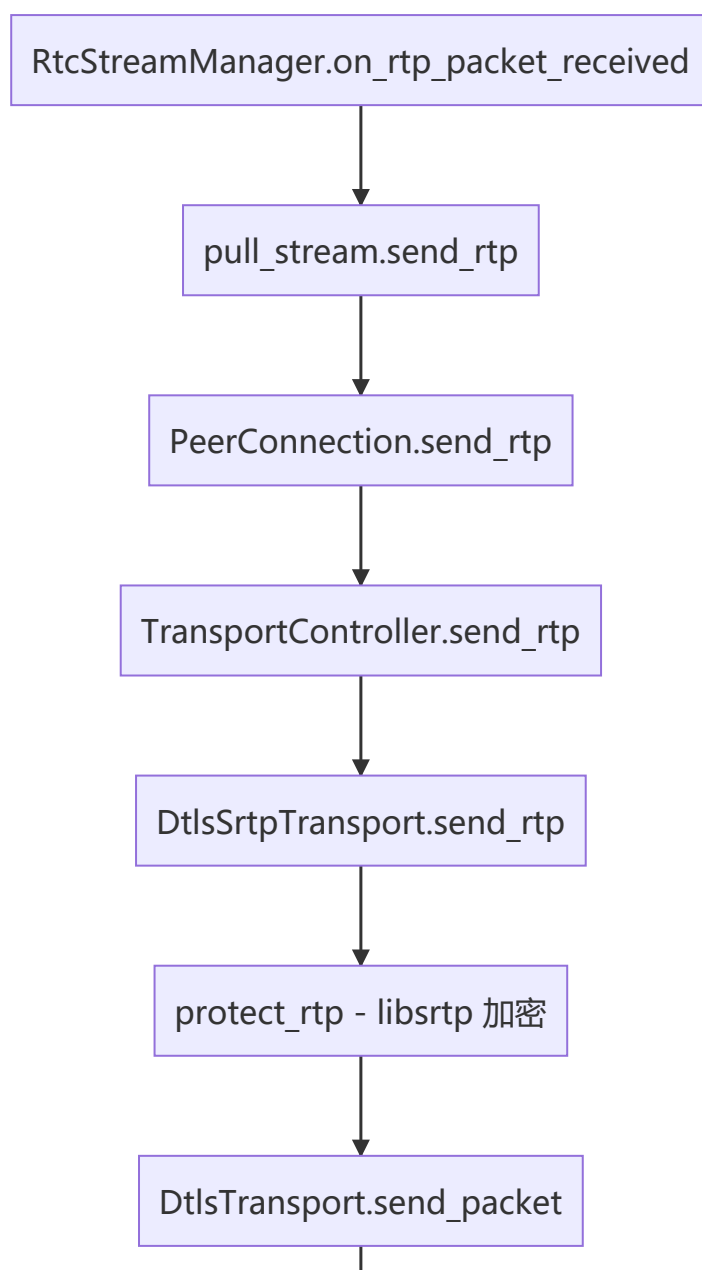


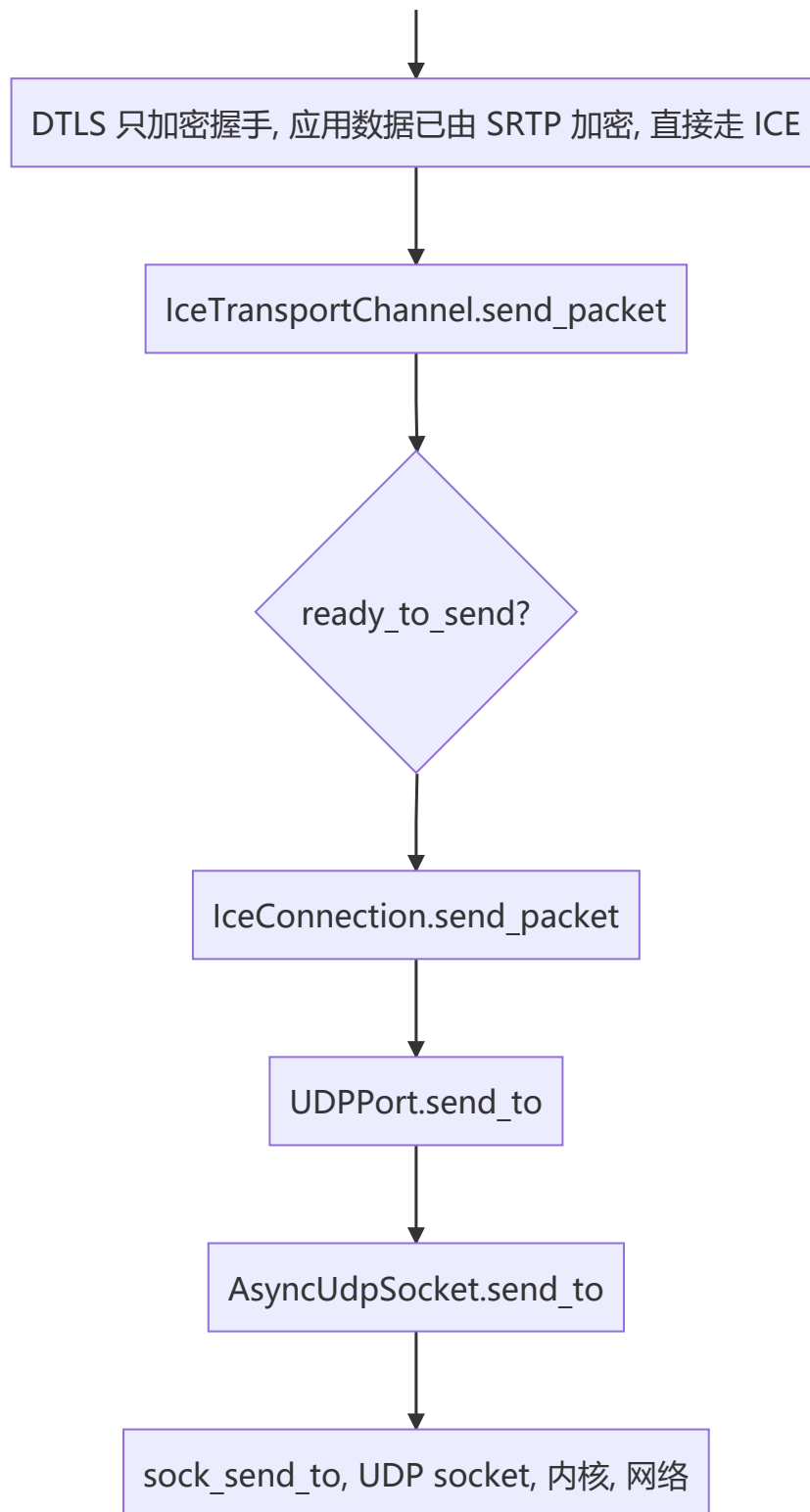
9.2 四个分拣层

层级	分拣点	判断依据	去向
第1层	<code>UDPPort::_on_read_packet</code>	是否有已知连接?	有 → <code>IceConnection</code> ; 无 → <code>get_stun_message</code>
第2层	<code>IceConnection::on_read_packet</code>	CRC32 fingerprint 校验	通过 → STUN 处理; 失败 → <code>signal_read_packet</code> 上交
第3层	<code>DtlsTransport::_on_read_packet</code>	<code>buf[0]</code> 在 20-63? + <code>buf[13]==1?</code>	DTLS → OpenSSL; 其他 → <code>signal_read_packet</code> 上交
第4层	<code>DtlsSrtpTransport::_on_read_packet</code>	<code>buf[1]</code> & 0x7F 在 [64,96)?	RTP → <code>unprotect_rtp</code> ; RTCP → <code>unprotect_rtcp</code>

9.3 发包容积链（完整）

加密 RTP 的发送路径从上到下贯穿整个栈：



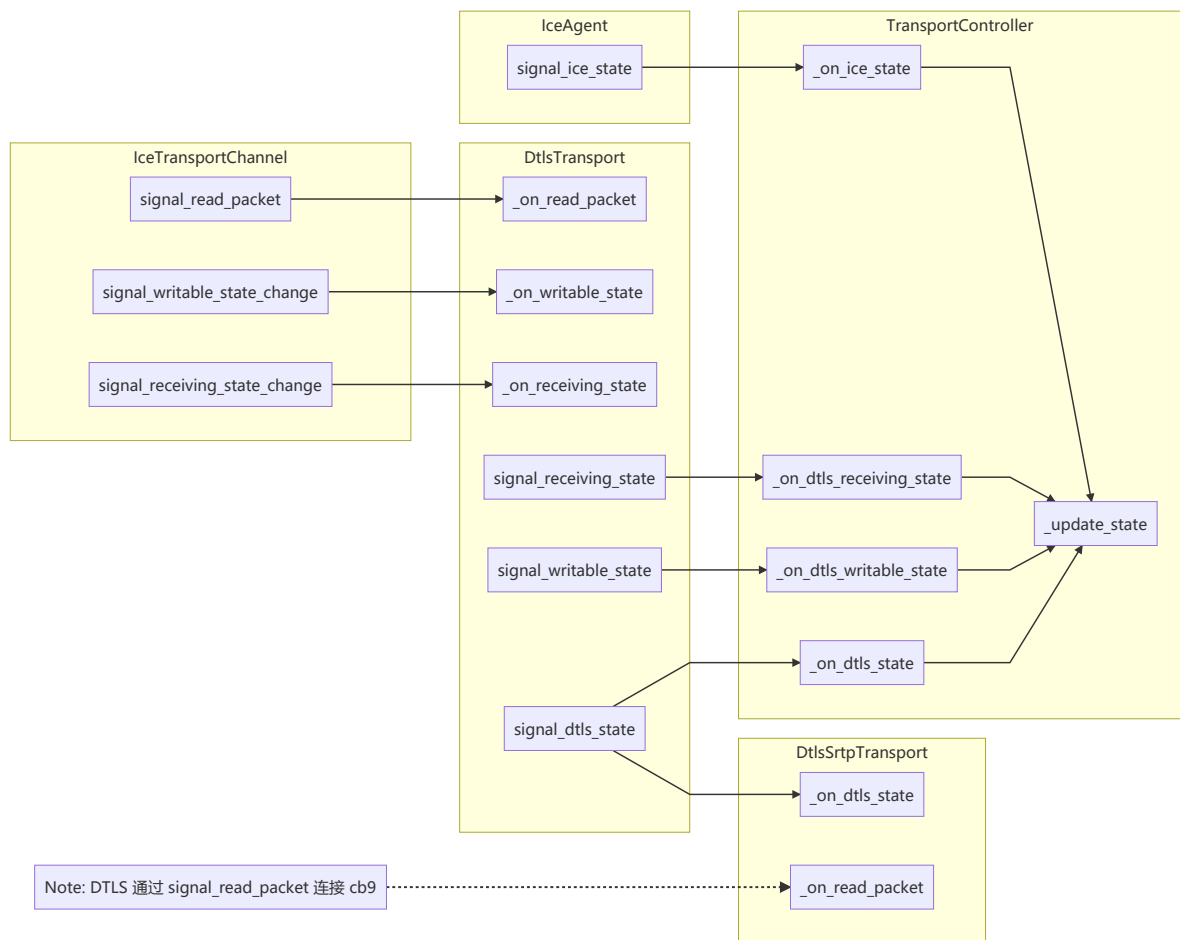


10. 信号链: ICE writable → DTLS → SRTP → RTP 就绪

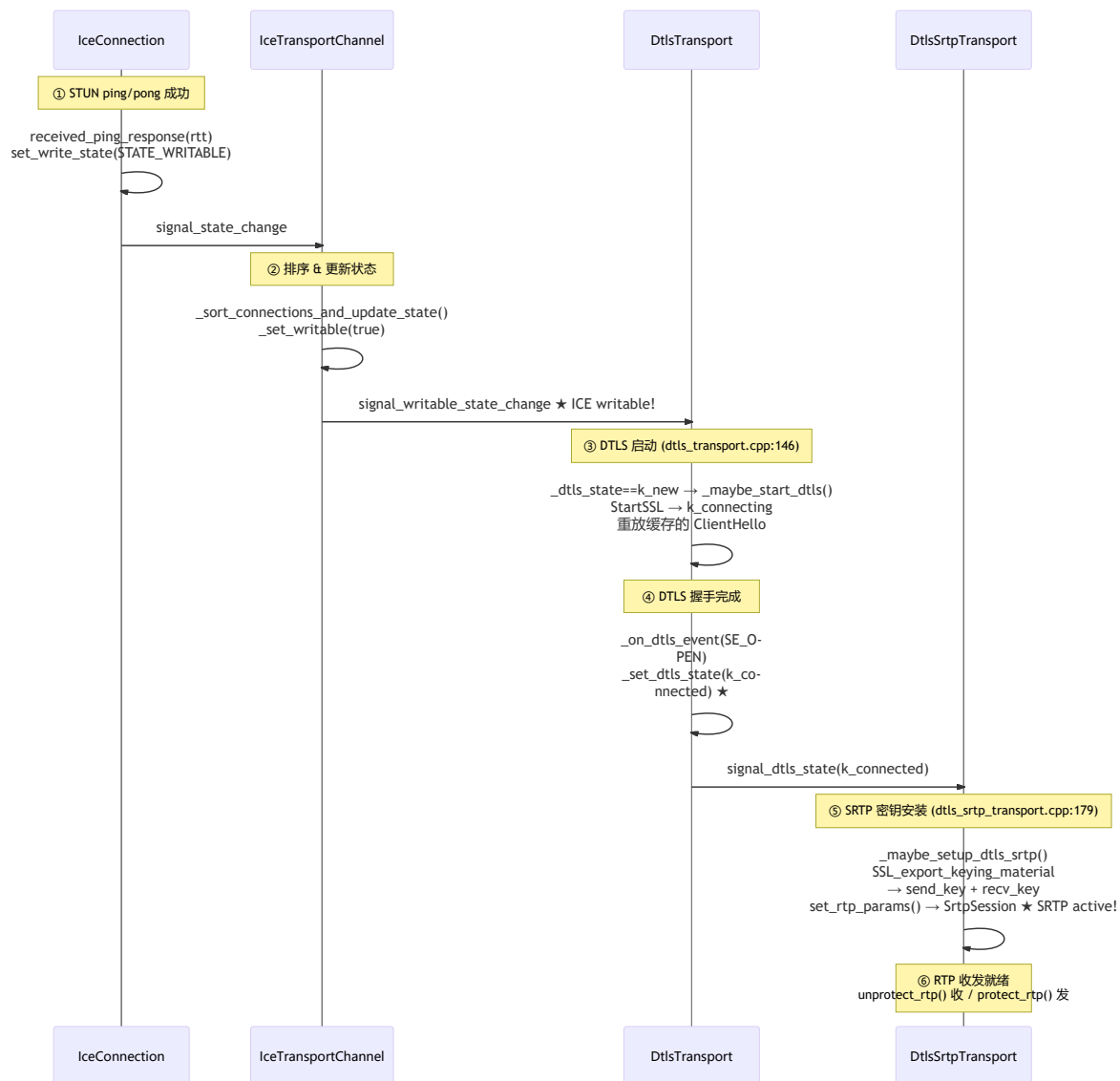
这是最具"多米诺骨牌效应"的一条链。

10.1 信号订阅全景

在 `TransportController::set_local_description()` 中建立:

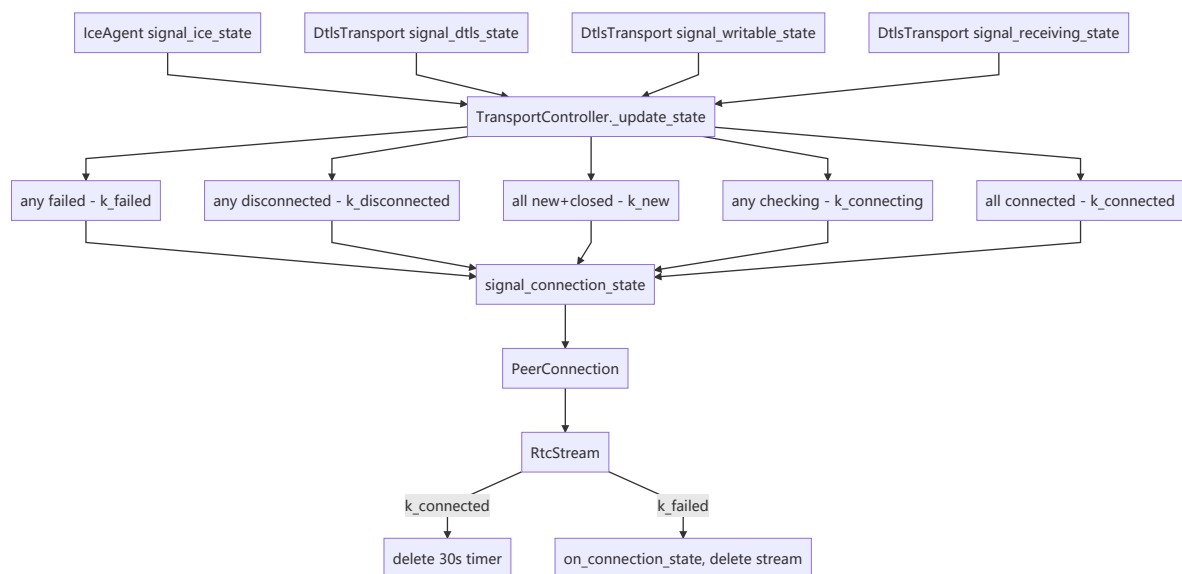


10.2 多米诺骨牌



10.3 PC 状态聚合

4 个信号源汇聚到 `TransportController::_update_state()`
(`src/pc/transport_controller.cpp:128-171`) :



11. RTP/RTCP 数据包处理

11.1 两级解复用

UDP 收包 → DtlsTransport (第一级) → DtlsSrtpTransport (第二级)

DTLS vs 非 DTLS RTP vs RTCP

第一级——`DtlsTransport::_on_read_packet()` (`dtls_transport.cpp:176-233`)：

判断	条件	去向
DTLS	<code>len >= 13 && buf[0] 在 20..63</code>	OpenSSL
RTP/RTCP	<code>len >= 12 && (buf[0] & 0xc0) == 0x80</code> 且 DTLS 已 connected	<code>signal_read_packet</code> 上交

- DTLS ContentType: 20=ChangeCipherSpec, 21=Alert, 22=Handshake, 23=ApplicationData
- RTP version bits (byte0 高 2 位) = 2 (二进制 10) —— SRTP 加密后版本位仍可见

第二级——`DtlsSrtpTransport::_on_read_packet()` (`dtls_srtp_transport.cpp:104-120`) 调用 `infer_rtp_packet_type()`:

```
// rtp_utils.cpp: infer_rtp_packet_type()
RtpPacketType infer_rtp_packet_type(ArrayView<const char> packet) {
    // ① 先检查 RTP
    if (is_rtp_packet(packet)) return k_rtp;
    // ② 再检查 RTCP
    if (is_rtcp_packet(packet)) return k_rtcp;
    // ③ 都不是
    return k_unknown;
}

// is_rtp_packet: len>=12 && version==2 && PT 不在 [64,96)
// is_rtcp_packet: len>=4 && (buf[1] & 0x7F) 在 [64,96)
```

PT 区分规则 (RFC 5761) : byte1 的低 7 位在 [64,96) 为 RTCP 保留范围, 其余为 RTP。

11.2 解密与转发链

```

DtlsSrtpTransport::_on_read_packet()
├─ k_rtp → _on_rtp_packet_received()
│   └─ unprotect_rtp() 原地解密，剥离尾部 auth tag
│       └─ signal_rtp_packet_received → TransportController →
PeerConnection
│
└─ k_rtcp → _on_rtcp_packet_received()
    └─ unprotect_rtcp() 原地解密，剥离 SRTCP index + auth tag
        └─ signal_rtcp_packet_received → TransportController →
PeerConnection

```

`SrtpSession` 方向隔离 (`srtp_transport.cpp`) :

Session	职责	libsrtp 方向
<code>_send_session</code>	加密发出	<code>ssrc_any_outbound</code> ，只允许 <code>protect_rtp/protect_rtcp</code>
<code>_recv_session</code>	解密收到	<code>ssrc_any_inbound</code> ，只允许 <code>unprotect_rtp/unprotect_rtcp</code>

11.3 RtcStreamManager 转发逻辑

```
// rtc_stream_manager.cpp:193-214

// RTP: push → pull 单向
void on_rtp_packet_received(stream, data, len) {
    if (k_push == stream->type()) {
        PullStream* pull = _find_pull_stream(stream->stream_name());
        if (pull) pull->send_rtp(data, len);
    }
}

// RTCP: 双向
void on_rtcp_packet_received(stream, data, len) {
    if (k_push == stream->type()) {
        PullStream* pull = _find_pull_stream(stream->stream_name());
        if (pull) pull->send_rtcp(data, len);    // push 的 RR/PLI → pull
    } else if (k_pull == stream->type()) {
        PushStream* push = _find_push_stream(stream->stream_name());
        if (push) push->send_rtcp(data, len);    // pull 的 PLI → push (请求 I 帧)
    }
}
```

RTCP 双向的必要性：拉流端发 PLI (Picture Loss Indication) 请求 I 帧，必须到达推流端；推流端发 SR (Sender Report) 让拉流端做音视频同步。

11.4 发包路径

```
pull_stream->send_rtp(data, len)
→ PeerConnection::send_rtp()           // hardcoded mid="audio"
→ TransportController::send_rtp("audio", ...)
→ DtlsSrtpTransport::send_rtp()
  └─ get_send_auth_tag_len() 预留 auth tag
  └─ protect_rtp() SRTP 加密
  └─ _rtp_dtls_transport->send_packet()
      → DtlsTransport::send_packet()    // 直接走 ICE，不经过 DTLS 加密
      → IceTransportChannel::send_packet()
      → _selected_connection->send_packet()
      → UDPPort::send_to() → UDP socket
```

12. PULL 流 + SSRC 透传

PushStream 和 PullStream 是两个完全独立的媒体栈实例，各有一套 ICE/DTLS/SRTP。两者的连接点只在 `RtcStreamManager` ——它从 push 端提取 SSRC 注入 pull 端 offer，并在运行时做 RTP/RTCP 应用层转发。

12.1 创建流程

```
signalingworker::_process_pull()
→ g_rtc_server->send_rtc_msg(msg)
→ CRC32(stream_name) 路由到同一 RtcWorker
→ RtcWorker::_process_pull()
→ RtcStreamManager::create_pull_stream()
```

`create_pull_stream()` 内部五步 (`rtc_stream_manager.cpp:52-82`) :

```
// ① 查找已有 PushStream
PushStream* push_stream = _find_push_stream(stream_name);
if (!push_stream) return -1; // 没推流就拉不了

// ② 从 push 的 remote SDP 提取 SSRC
std::vector<StreamParams> audio_source, video_source;
push_stream->get_audio_source(audio_source);
push_stream->get_video_source(video_source);

// ③ 移除旧的 pull (同一 stream_name 只有一个)
_remove_pull_stream(uid, stream_name);

// ④ 新建 PullStream, 注入 push 的 SSRC
PullStream* stream = new PullStream(_el, _allocator.get(), uid, stream_name,
...);
stream->add_audio_source(audio_source);
stream->add_video_source(video_source);
stream->start(certificate);

// ⑤ 生成带 SSRC 的 offer, 存 map
offer = stream->create_offer();
_pull_streams[stream_name] = stream;
```

12.2 PushStream 如何提取 SSRC

`push_stream.cpp:36-61` —— `get_audio_source()` / `get_video_source()` 委托给 `_get_source(mid, source)` :

```
auto remote_desc = _pc->remote_desc(); // push 端收到 answer 后存入的
auto content = remote_desc->get_content(mid); // "audio" / "video"
source = content->streams(); // vector<StreamParams>
```

`StreamParams` (`pc/stream_params.h`) 承载:

```
struct StreamParams {
    std::string id; // track id
    std::vector<uint32_t> ssrcs; // 推流端真实 SSRC
    std::vector<SsrcGroup> ssrc_groups; // FID 等分组
    std::string cname; // CNAME
    std::string stream_id; // msid
};
```

这些数据来自 push 客户端 ANSWER SDP 的 `a=ssrc:` 行解析 (`peer_connection.cpp:215-273` 的 `parse_ssrc_info()`) 。

12.3 PullStream 的 offer 怎么用这些 SSRC

```
// pull_stream.cpp:22-33
options.send_audio = _audio;      // sendonly
options.recv_audio = false;
options.send_video = _video;
options.recv_video = false;
```

PeerConnection::create_offer() 对 send 方向 (line 115-119, 127-133) :

```
if (options.send_audio) {
    for (auto stream : _audio_source) {      // push 端的 SSRC
        audio->add_stream(stream);           // 注入到 SDP m=audio section
    }
}
```

SDP 序列化 (session_description.cpp:252) 写出 push 端原始 SSRC:

```
a=ssrc-group:FID 67890 67891
a=ssrc:67890 cname:clientvideoCname
a=ssrc:67890 msid:stream1 video_track
```

关键约束: SFU 不转码，只做 SRTP 解密→重加密。SSRC 是 RTP 包头核心标识，一旦改写，拉流端无法把收到的 RTP 与 SDP 声明的流对应。

12.4 PushStream vs PullStream 对比

维度	PushStream	PullStream
SDP direction	a=recvonly	a=sendonly
offer 中 SSRC	无	push 端的原始 SSRC
send_audio/video	false	true
recv_audio/video	true	false
数据方向	从 push 客户端接收	发送给 pull 客户端
媒体栈	独立 ICE/DTLS/SRTP	独立 ICE/DTLS/SRTP

13. STOP 与资源清理链

5.13 两个清理入口

入口	触发条件	调用栈
on_connection_state(k_failed)	ICE/DTLS 状态机检测到失败	在 ICE ping timer 回调栈内
on_stream_exception()	30 秒 ICE 超时定时器触发	独立 timer 回调，不在 ping 栈内

两者都会调用 `_remove_push_stream(stream)` / `_remove_pull_stream(stream)`，收敛到同一个 `delete` 路径。

5.14 UID 校验 + delete

```
void RtcStreamManager::_remove_push_stream(uint64_t uid, const string&
stream_name) {
    PushStream* push_stream = _find_push_stream(stream_name);
    if (push_stream && uid == push_stream->get_uid()) {
        _push_streams.erase(stream_name); // 先从 map 移除
        delete push_stream;              // 再销毁
    }
}
```

5.15 析构瀑布

`delete push_stream` 触发的完整析构路径：

```
~PushStream() // 日志
└→ ~RtcStream()
    └ delete_timer(_ice_timeout_watcher) // 取消 30s 超时
      └ _pc->destroy() // ★ 不直接 delete

_pc->destroy():
└→ create_timer(destroy_timer_cb, 10ms) // 一次性定时器
└→ return // 当前栈退出

[10ms 后, 事件循环空闲]
destroy_timer_cb():
└→ delete pc

~PeerConnection(): // private 析构
└→ ~unique_ptr<TransportController>

~TransportController():
└ for each DtlsSrtpTransport: delete
  └→ ~SrtpTransport()
    └ reset _send_session (SrtpSession) // libsrtp 发送 session
      └ reset _recv_session (SrtpSession) // libsrtp 接收 session
└ for each DtlsTransport: delete
  └→ ~SSLStreamAdapter() // OpenSSL DTLS 上下文
└ delete _ice_agent

~IceAgent():
└→ for each IceTransportChannel: delete

~IceTransportChannel():
└ delete_timer(_ping_watcher) // 停 ping 定时器
└ for each IceConnection: conn->destroy()
  └→ signal_connection_destroy → IceController 清理引用
  └→ delete this
└ for each UDPPort: delete port
  └ ~UDPPort():
    └ close(_socket) // ★ 关闭 UDP socket fd
      └ ~AsyncUdpSocket()
        └ delete_io_event(_socket_watcher) // 从 libev 移除
```

```
|           └ delete[] _buf           // 释放 1500 字节缓冲
└ _ice_controller.reset()           // unique_ptr 销毁
```

5.16 为什么需要 10ms 延迟析构

```
ICE ping timer (48ms) → _on_check_and_ping()
→ _update_connection_states()
→ conn->update_state(now)
→ set_write_state(STATE_WRITE_TIMEOUT)
→ signal_state_change
→ _on_connection_state_change
→ _sort_connections_and_update_state
→ _compute_ice_transport_state → k_failed
→ signal_ice_state_change
→ IceAgent::_update_state → signal_ice_state
→ TransportController::_on_ice_state → _update_state →
k_failed
→ signal_connection_state → PeerConnection → RtcStream
→ _listener->on_connection_state(k_failed)
→ _remove_push_stream → delete push_stream
→ ~RtcStream → _pc->destroy()
→ create_timer(10ms) → return
← 回到 _on_check_and_ping() ★ 如果没有延迟，
_ice_controller 已被 delete!
→ _ice_controller->select_connection_to_ping() ★ nullptr dereference →
coredump
```

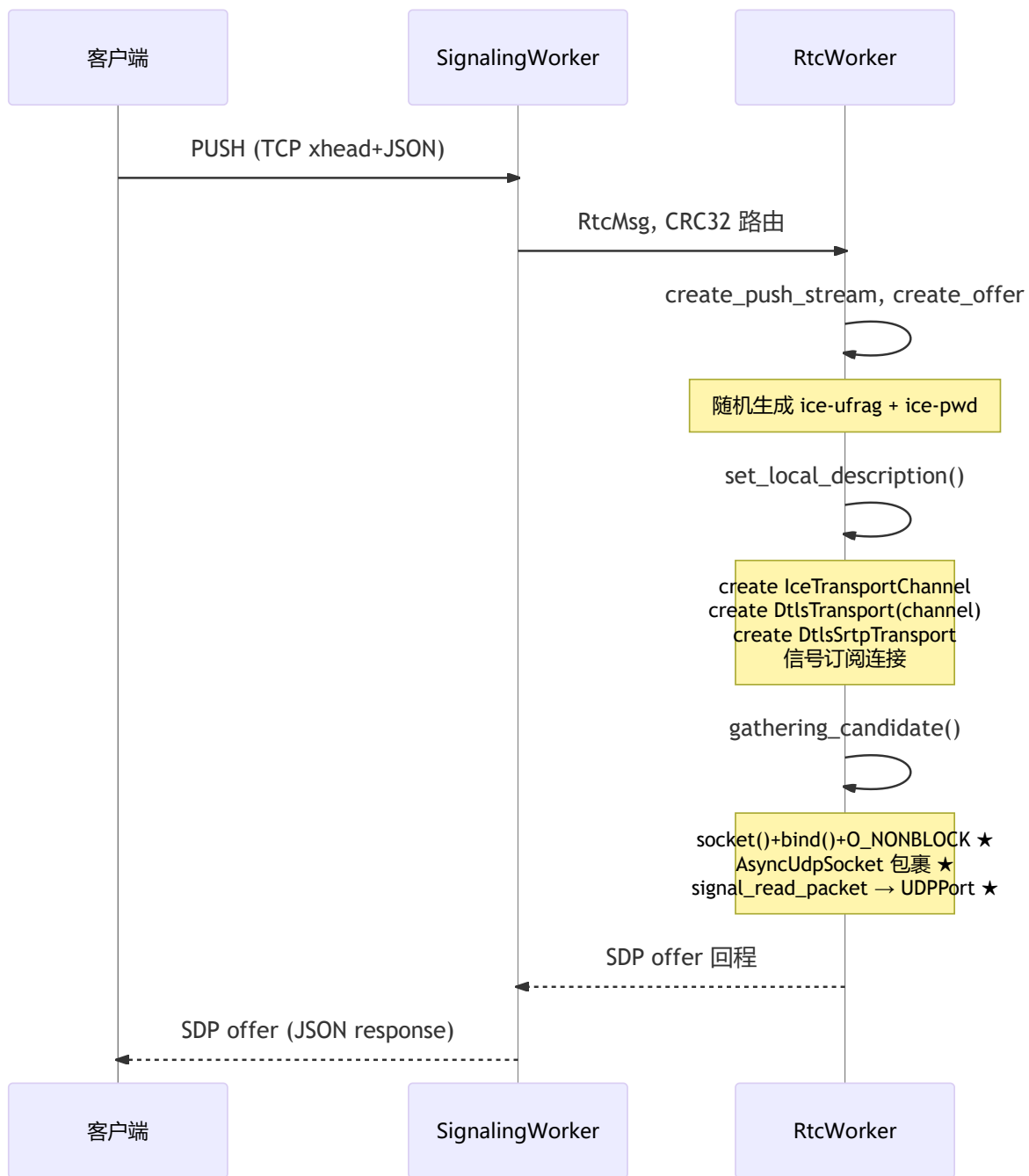
10ms: 足够当前 event loop 迭代完成并返回 libev，但人眼无感知。~PeerConnection() 设为 private + destroy_timer_cb 为 friend，编译期强制走延迟析构路径。

5.17 STOP 命令 vs 异常清理对比

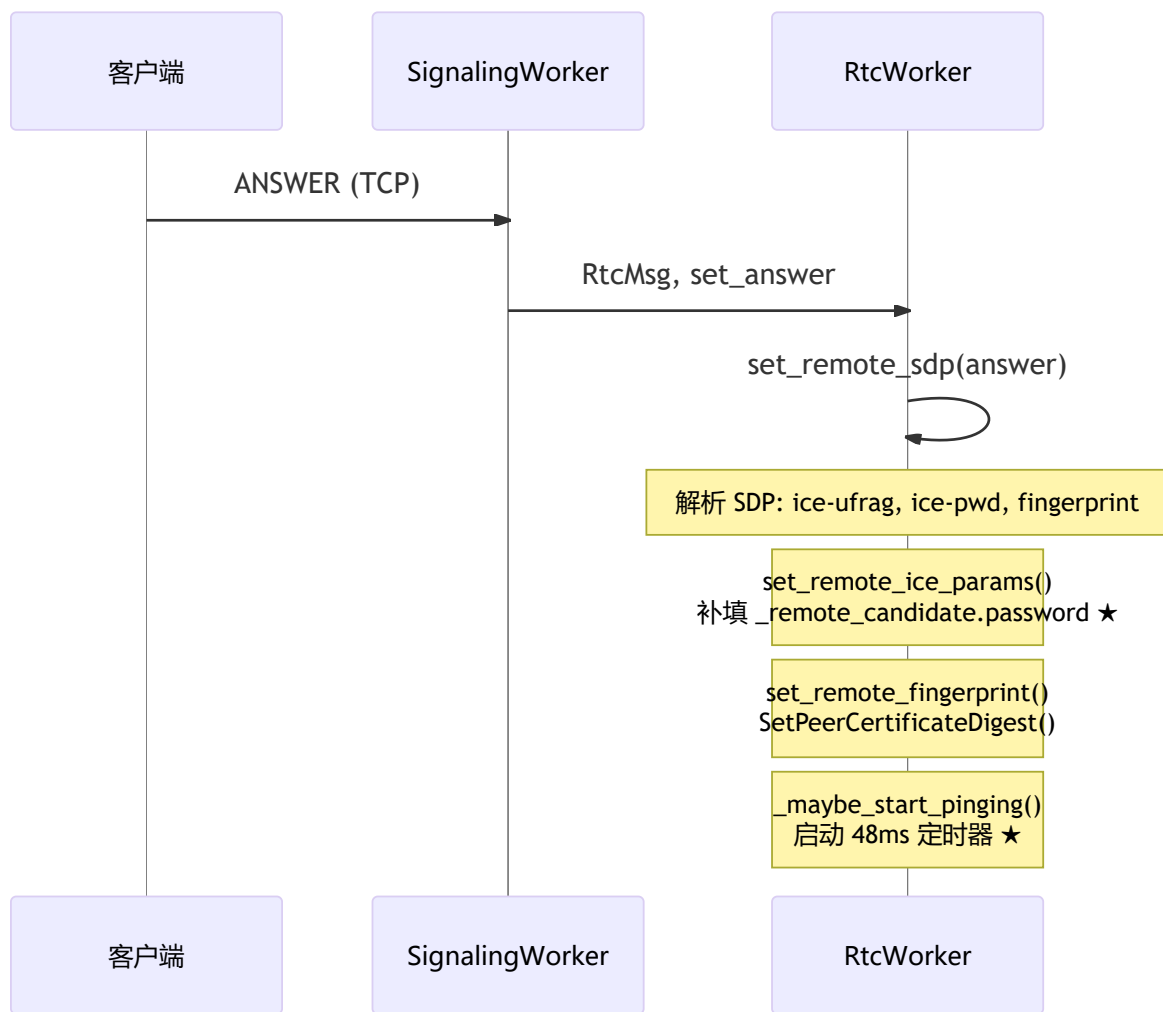
维度	STOP_PUSH / STOP_PULL	on_connection_state(k_failed)	on_stream_exception
触发	客户端主动发 STOP	ICE 连接全断	30s 超时
调用链	SignalingWorker → RtcServer → RtcWorker	ICE ping timer 回调栈内	独立 timer
响应	返回 JSON {errno:0}	无响应	无响应
UID 校验	有 (外部传入)	有 (从 stream 对象取)	有 (从 stream 对象取)
收敛点	_remove_push/pull_stream(uid, name)	_remove_push/pull_stream(stream)	_remove_push/pull_stream(stream)

14. 完整生命周期时间线

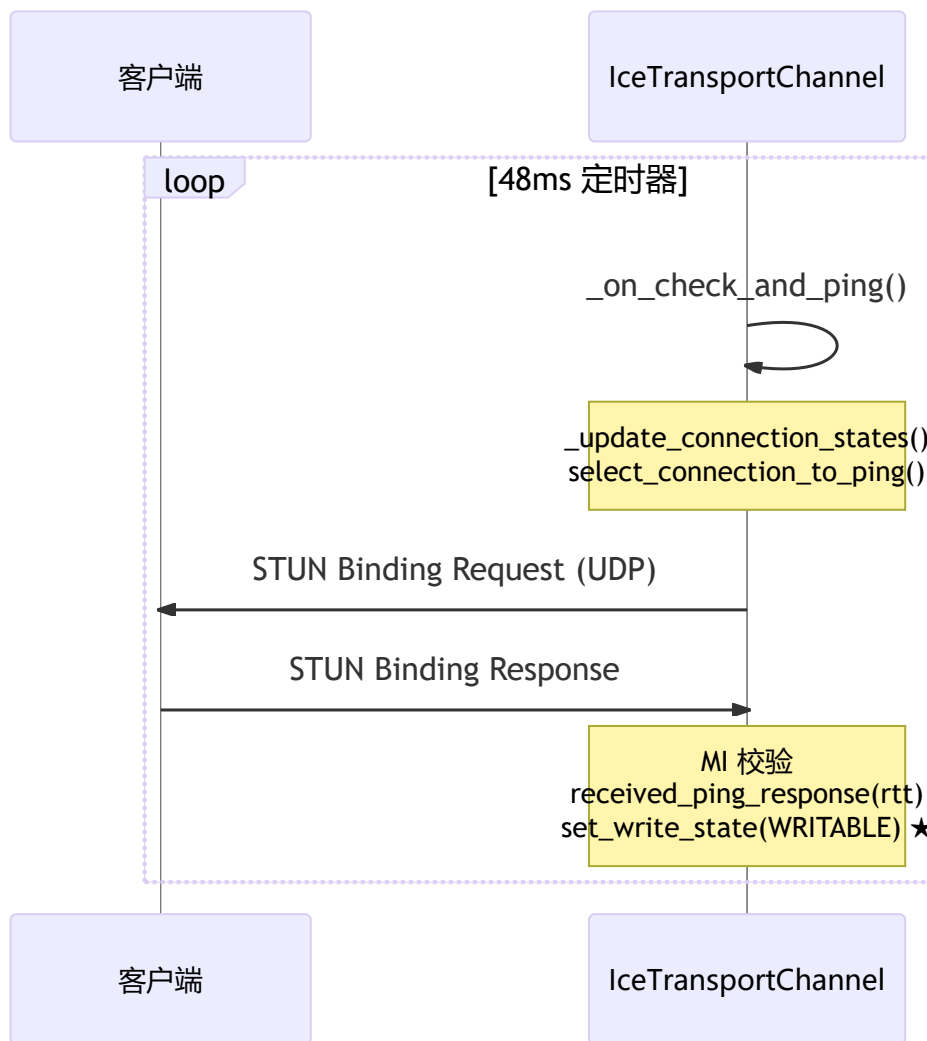
Phase 1 — 媒体栈诞生 (set_local_description)



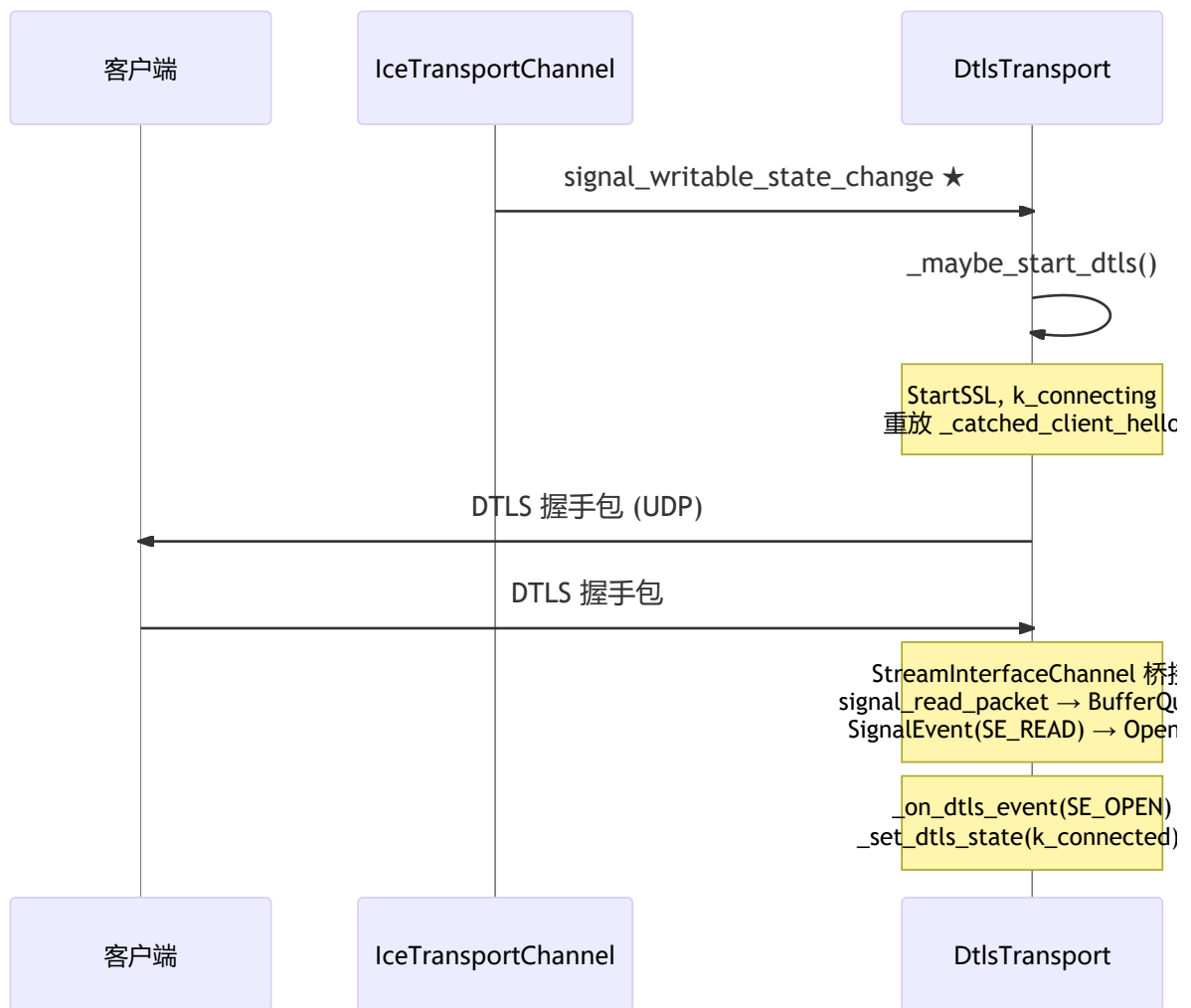
Phase 2 — 远端信息注入 (set_remote_description)



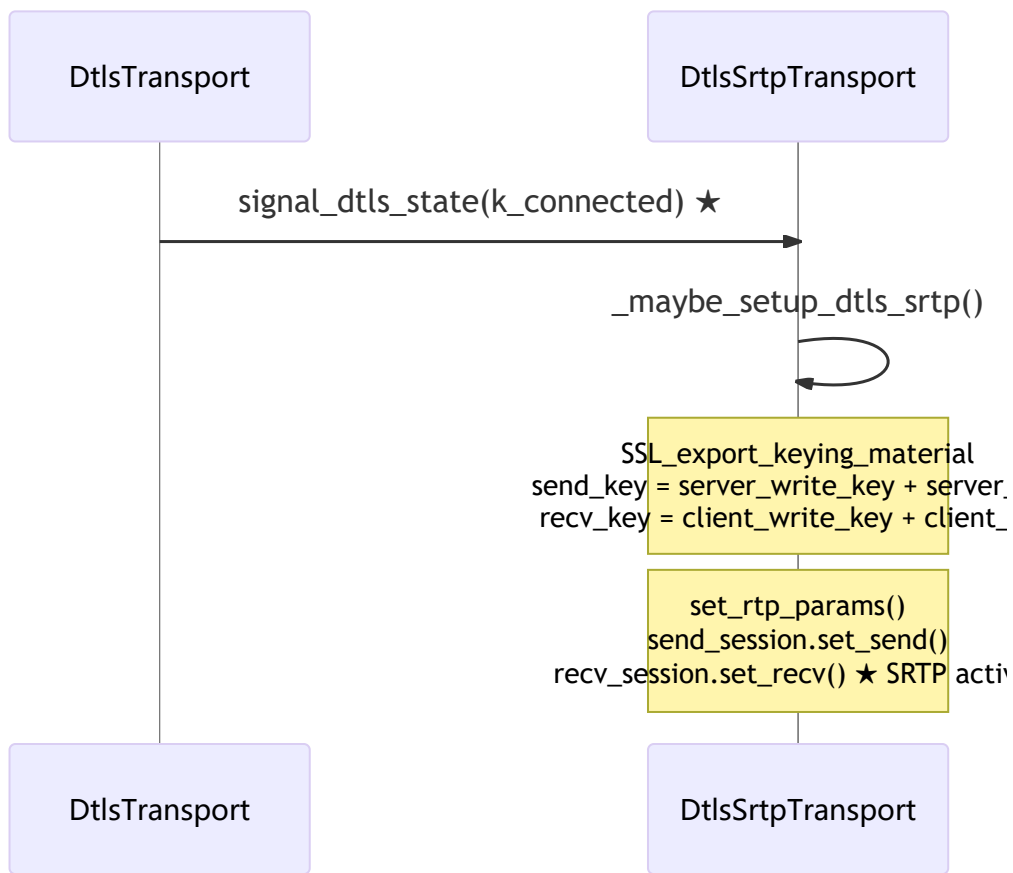
Phase 3 — ICE 连通性检查



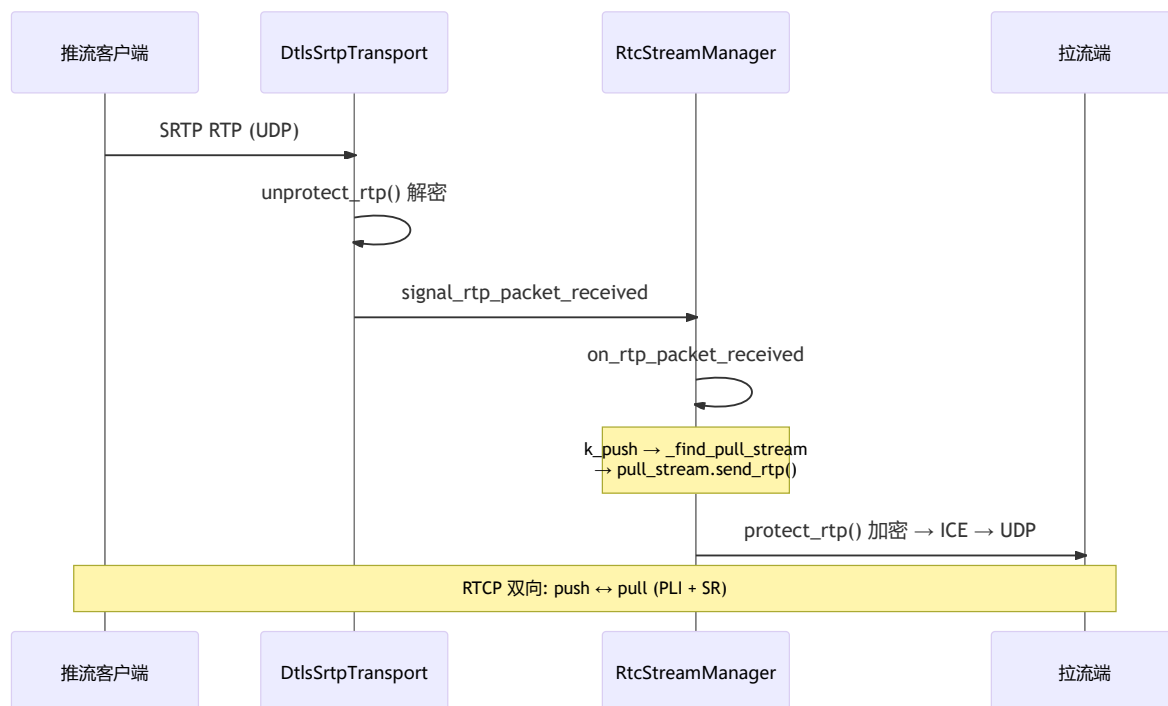
Phase 4 — DTLS 握手



Phase 5 — SRTP 密钥安装



Phase 6 — RTP 流转发



15. FAQ：你应该知道但可能没问的问题

15.1 STUN MESSAGE-INTEGRITY 的密码使用规则："发给谁，就用谁的密码"

STUN Binding Request 的 MI 用**接收方**的密码计算，Binding Response 复用**同一个**密码。双方各有自己的 ice-pwd，形成完美的对称：

场景	方向	密码	代码位置
客户端 → SFU 的 Request	收到	<code>_ice_params.ice_pwd</code> (SFU 密码)	<code>udp_port.cpp:195</code> 验证
SFU → 客户端的 Response	发出	<code>_port->ice_pwd()</code> = 同上	<code>ice_connection.cpp:62</code> 构造
SFU → 客户端的 Request (ping)	发出	<code>remote_candidate().password = _remote_ice_params.ice_pwd</code> (客户端密码)	<code>stun_request.cpp:131</code> 构造
客户端 → SFU 的 Response	收到	<code>_remote_candidate.password</code> = 同上	<code>ice_connection.cpp:124</code> 验证

规则：Request 发给谁就用谁的密码，Response 跟 Request 用同一个密码。

另外，远端密码也是 `_is_pingable` 的门控条件——**密码非空才说明 ANSWER 已到达、对端身份已确认**，此时连接才可 ping。

15.2 `DtlsTransport::send_packet()` 为什么绕过 DTLS 加密？

```
// src/pc/dtls_transport.cpp:554
int DtlsTransport::send_packet(const char* data, size_t len) {
    return _channel->send_packet(data, len); // 直接走 ICE!
}
```

因为调用者是 `DtlsSrtpTransport::send_rtp()`——数据在到达这里之前**已经被 SRTP 加密过了**。DTLS 只加密握手消息（通过 OpenSSL 内部 BIO write），应用数据不需要经过 DTLS 层。这是 DTLS-SRTP 标准（RFC 5764）的设计：**DTLS 握手 → 密钥导出 → SRTP 接管应用数据**。

15.3 30s ICE 超时和 PC 状态 `k_connected` 的关系

```
// src/stream/rtc_stream.cpp:72-78
void ice_timeout_cb(...) {
    if (stream->_state != PeerConnectionState::k_connected) {
        stream->_listener->on_stream_exception(stream); // 删除流
    }
}
```

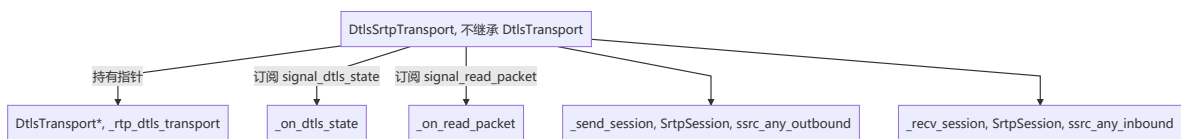
30s 一次性定时器在 `start()` 时创建。如果 30s 内 PC 到达 `k_connected`，定时器被删除。如果没到达，触发异常清理。这是 ICE/DTLS 握手的总超时兜底。

15.4 PushStream 和 PullStream 的 create_offer() 本质区别

```
// push_stream.cpp: recvonly → SDP 不含 SSRC  
// pull_stream.cpp: sendonly → SDP 含 push 端的 SSRC (透传)
```

虽然方向不同，但它们调用的是**同一个** `PeerConnection::create_offer()` → **同一个** `TransportController::set_local_description()`。这意味着 PushStream 和 PullStream 各自拥有独立的 ICE channel、UDP socket、DTLS 上下文、SRTP session。两者之间的 RTP 数据通过 `RtcStreamManager` 做应用层转发：push 收（解密）→ pull 发（加密）。

15.5 DtlsTransport 和 DtlsSrtpTransport 的"组合优于继承"关系



`DtlsTransport` 不知道自己上面有 `DtlsSrtpTransport`。两者可独立测试。

`_maybe_setup_dtls_srtp` 被 `_on_dtls_state`（握手完成信号）和 `set_dtls_transport`（兜底）双触发，内部用 `is_srtp_active() || !is_dtls_writable()` 做幂等——"信号驱动 + 手动兜底"解决时序竞争。

15.6 PeerConnection::destroy() 延迟析构

```
// src/pc/peer_connection.cpp:88-96  
void PeerConnection::destroy() {  
    _destroy_timer = _el->create_timer(destroy_timer_cb, this, false);  
    _el->start_timer(_destroy_timer, 10000); // 10ms 后 delete this  
}
```

问题场景：ICE timer → `_on_check_and_ping()` → `_update_connection_states()` → conn timeout → signal 上报 → delete stream → 析构 IceController。但 `_on_check_and_ping` 返回后还要调用 `_ice_controller->select_connection_to_ping()`——如果在回调栈内同步析构，`_ice_controller` 已被释放 → coredump。

延迟 10ms 确保当前 event loop 迭代完整退出后才析构。`~PeerConnection()` 设为 private，编译期拦截所有直接 `delete`。

15.7 StreamInterfaceChannel 的 BIO 桥接

```
// src/pc/dtls_transport.cpp:62-84  
class StreamInterfaceChannel {  
    BufferQueue _packets; // 容量 2，每个最大 2048 字节  
  
    // ICE 收包 → 写入 BufferQueue → 唤醒 OpenSSL
```

```

bool on_received_packet(const char* data, size_t size) {
    _packets.WriteBack(data, size, nullptr);
    SignalEvent(this, rtc::SE_READ, 0); // "叫 OpenSSL 来读"
}

// OpenSSL 调用 Read() → 从 BufferQueue 取
rtc::StreamResult Read(void* buffer, size_t buffer_len, size_t* read, ...) {
    if (!_packets.ReadFront(buffer, buffer_len, read))
        return rtc::SR_BLOCK; // 队列空 → 阻塞 OpenSSL
    return rtc::SR_SUCCESS;
}

// OpenSSL 调用 write() → 直接经 ICE 发出
rtc::StreamResult write(const void* data, size_t data_len, ...) {
    _channel->send_packet((const char*)data, data_len);
}
};

```

两个方向的 `SignalEvent` 含义不同：我们发 `SE_READ` 是"叫 OpenSSL 来读 BufferQueue"，OpenSSL 发 `SE_OPEN` 是"握手完成"、发 `SE_READ` 是"解密数据就绪"。谁点火、谁接收是关键区分。

16. DTLS Transport 深挖

16.1 定位：被动的中间层

`DtlsTransport` 自己不主动做任何事。它坐在 ICE channel 旁边，靠三个信号驱动：

```

// dtls_transport.cpp:131-138
DtlsTransport::DtlsTransport(IceTransportChannel* channel) : _channel(channel) {
    _channel->signal_read_packet.connect(this, &DtlsTransport::_on_read_packet);
    _channel->signal_writable_state_change.connect(this,
&DtlsTransport::_on_writable_state);
    _channel->signal_receiving_state_change.connect(this,
&DtlsTransport::_on_receiving_state);
}

```

构造函数不创建 OpenSSL 上下文，不启动握手。只保存指针 + 订阅信号。

16.2 三个开工条件——谁最后到谁触发

`DtlsTransport` 正式启动 DTLS 握手需要三个条件同时满足：

条件	含义	何时就绪
<code>_dtls</code> 已创建	OpenSSL DTLS 上下文存在	<code>_setup_dtls()</code> 被调用后
<code>_remote_fingerprint_value</code> 非空	客户端 DTLS 证书指纹已知	ANSWER SDP 到达后
<code>_channel->writable()</code> 为 true	ICE 通道可发送数据	STUN ping/pong 成功后

三个条件由三个独立事件交付，以任意顺序到达：

事件 A: 收到 DTLS ClientHello (UDP)

```
→ _on_read_packet(k_new)
→ _cached_client_hello 缓存
→ _setup_dtls() ← 条件: _local_certificate 已设置
    OpenSSL 上下文创建 (_dtls 就绪 ✓)
    fingerprint 空的 → SetPeerCertificateDigest 跳过
→ _maybe_start_dtls()
    → _dtls ✓, fingerprint X, writable X → 不启动
```

事件 B: ANSWER 到达 (TCP)

```
→ set_remote_fingerprint()
→ 存 _remote_fingerprint_alg/value (fingerprint 就绪 ✓)
→ 如果 _dtls 还没创建 → _setup_dtls()
→ 如果 _dtls 已存在 (事件A先到) → 补调 SetPeerCertificateDigest()
→ _maybe_start_dtls()
    → _dtls ✓, fingerprint ✓, writable X → 不启动
```

事件 C: ICE writable

```
→ _on_writable_state(k_new)
→ _maybe_start_dtls()
    → _dtls ✓, fingerprint ✓, writable ✓ → StartSSL! ★
```

最后一个到达的事件触发 StartSSL。这种设计是 DTLS 层与 ICE 层异步解耦的核心——DTLS 不关心 ICE 何时通、SDP 何时到，它只等三者齐全。

16.3 _setup_dtls: OpenSSL 上下文的创建

`_setup_dtls()` (`dtls_transport.cpp:243-287`) 创建完整的 OpenSSL DTLS 服务端上下文，可被多次调用 (`_dtls` 不存在时才创建新的)：

```
bool DtlsTransport::_setup_dtls() {
    // ① 创建 BIO 适配器: ICE ↔ OpenSSL
    auto downward = std::make_unique<StreamInterfaceChannel>(_channel);
    _downward = downward.get();

    // ② 创建 SSLStreamAdapter, 移入 downward (BIO 层)
    _dtls = SSLStreamAdapter::Create(std::move(downward));
    _dtls->SetIdentity(_local_certificate->identity()->Clone());
    _dtls->SetMode(SSL_MODE_DTLS);
    _dtls->SetMaxProtocolVersion(SSL_PROTOCOL_DTLS_12);
    _dtls->SetServerRole(SSL_SERVER); // ★ SFU 是 DTLS 服务端

    // ③ 订阅 OpenSSL 回调
    _dtls->SignalEvent.connect(this, &_on_dtls_event); //
    SE_OPEN/SE_READ/SE_CLOSE
    _dtls->SignalSSLHandshakeError.connect(this, &_on_dtls_handshake_error);

    // ④ 设置对端指纹 (如果已从 ANSWER 中获得)
    if (_remote_fingerprint_value.size()) {
        _dtls->SetPeerCertificateDigest(alg, data, size);
    } // 否则留空, 等 set_remote_fingerprint() 补调

    // ⑤ 设置 SRTP 密码套件 (DTLS-SRTP 扩展)
    _dtls->SetDtlsSrtpCryptoSuites(_srtp_ciphers);

    // ⑥ 尝试启动
```

```

    _maybe_start_dtls();
    return true;
}

```

关键：SFU 作为 `SSL_SERVER`（DTLS 服务端），客户端是 DTLS 客户端（发起 ClientHello）。

16.4 `_maybe_start_dtls`：条件启动

```

void DtlsTransport::_maybe_start_dtls() {
    if (_dtls && _channel->writable()) {           // 两个硬条件
        if (_dtls->startSSL()) {                   // OpenSSL 启动握手
            _set_dtls_state(k_failed);
            return;
        }
        _set_dtls_state(k_connecting);
        if (_cached_client_hello.size()) {
            _handle_dtls_packet(_cached_client_hello); // 重放缓存的
ClientHello
            _cached_client_hello.clear();
        }
    }
}

```

不一定每次都成功——如果 ICE 没 writable 或 `_dtls` 还没创建，静默返回。设计上允许被多次调用，谁最后一个拿到条件的谁触发。

16.5 状态机

```

k_new  —_maybe_start_dtls(), startSSL→ k_connecting
k_connecting —_on_dtls_event(SE_OPEN)→ k_connected
k_connecting —_on_dtls_handshake_error→ k_failed
k_connected —_on_dtls_event(SE_CLOSE)→ k_closed（正常关闭）
k_connected —_on_dtls_event(SE_CLOSE + error)→ k_failed（异常关闭）

```

终态（`k_failed` / `k_closed`）无恢复机制——进入后只能等上层 STOP 或 `k_failed` 销毁。

writable 的生命周期：`k_new` 时 mirror ICE writable，`k_connected` 后由 DTLS 自己管理：

```

// _on_writable_state(k_new)
→ _set_writable_state(channel->writable()) // 透传 ICE 状态

// _on_dtls_event(SE_OPEN)
→ _set_writable_state(true)                // DTLS 接管

// _on_dtls_event(SE_CLOSE)
→ _set_writable_state(false)               // DTLS 断开

```

后续待深挖：StreamInterfaceChannel（OpenSSL↔ICE 的 BIO 桥接）、`_handle_dtls_packet`（DTLS Record 校验与注入）、`_on_dtls_event SE_READ` 的解密循环。

